# Verifiability Is a Spectrum: A Single-Auditor, Artifact-Only Audit of Google's Sealed-Hardware Federated Learning Pipeline

Anonymous Author(s)

## Abstract

Google's federated learning (FL) system—which trains inside Trusted Execution Environments (TEEs) and is deployed in production for Gboard next-word models—claims that encrypted uploads are "decrypted only inside sealed hardware running publicly listed code" and that its central differential-privacy guarantees are externally verifiable. We ask what "externally verifiable" means operationally: for each externally-checkable claim, can an outsider with no device, no credentials, and no Google contact actually verify it from public artifacts? We develop a seven-dimension audit framework (attestation binding, code transparency, policy expressiveness, key lifecycle, firmware/boot chain, side-channel surface, operator-insider residual), induced from this deployment's own claim chain and exercised once on it, score each checkable claim as *verified*, *partially checkable*, or *trust-required*, and apply it in a worked audit grounded in public artifacts. Of the 23 vendor-statement rows in our condensed matrix, 2 score *verified*, 10 *partially checkable* (five executed-qualified—one an executed observation of bounded historical retrievability—four by inspection, one partially executed) and 11 *trust-required*. The matrix's four remaining rows are not vendor statements: two score disclosures whose content is the statement itself (both *verified* as stated), one scores an audit-derived dependency no vendor statement asserts (row C3c, *trust-required* on the evaluated routes of §4.4), and one records a scope limit rather than a claim (row G4; obligations with no checkable claim or artifact—DP output gating among them—have no row, §3). Over all 27 rows the condensed totals are 4 *verified* (two executed artifact verifications; two statements verified as stated), 10 *partially checkable* and 13 *trust-required*—all of them a single auditor's scores (§3 reports the reliability exercises and their limits). That distribution indexes the paper's main result—the residual-trust enumeration of §5, the row-by-row account of what an outsider cannot decide and what would decide it—rather than standing as the result itself. The *verified* count is 3–5 across the envelope—the declared domain of row constructions, strict scoring variants, and the claim-interpretation readings of §5 (the strict C1/C2 line reads 4/8/15; the B1/B2 merge of §5, outside the declared domain, would extend the range to 2–5), of the published matrix at its post-amendment state (4 under ours—the pre-amendment snapshot reads 3/10/12 over 25 rows, §5; one of the four amendment-raised, a replication awaiting blinded repetition, §4); the crossing is computed in §5, and the 29-row matrix is Appendix B. None of the four *verified* scores attests runtime behaviour: two are executed artifact verifications (the transparency-log chain for the exemplar, verified end to end including the log's own signatures; and the public source and build recipe—the recipe at digest equality, over provenance statements whose signatures we did not independently verify), and two are verified statements whose claims are the statements themselves (a stated freshness window; an acknowledged side-channel limitation). We replicated the guide's worked example end to end—transparency log, endorsement, binary, build provenance (at digest equality), source—recomputing every hash; the chain holds for the exemplar, the log's signatures included; A1's V is amendment-raised (§4). A later census of all 115 log entries sharing the exemplar's signing key byte-verified 110 endorsed payloads (the 5 earliest no longer served). Extended to every endorsement key we could reconstruct—19 keys, 509,569 entries, as returned by the v1 index endpoint (removed in v2, §4.4)—the census shows what the keys have endorsed *is* enumerable (row C3: P, not V—also amendment-raised, §4). Whether the vendor's key inventory is itself complete is a separate, trust-required claim; whether the index returns are complete against the log's tree is a third (row C3c); and payload readability is bounded by retention. Twenty SLSA v1 provenance statements from twenty source commits assert the identical subject digest for the KMS binary—consistent with toolchain determinism, but also with build caching: input-sensitivity is not established (14 of the 20 touch neither the KMS tree nor the shared module file; §4.1). The reproducibility claim has an executed check—an exploratory rescue (§4.3), not a frozen-protocol confirmation: the project's CI build invocation reproduced the endorsed digest bit-for-bit for that one binary, at the exemplar-era commit. (The amendment-raised results arrived by flagged routes, detailed in §4.) The audit guide's February-2025 policy worked example is already unserved from the store; a substantial share of endorsed payloads is unserved (the exemplar key's five unserved are its earliest entries); and the key inventory on which enumeration rests shows drift—two guide fingerprints match no current key. Live access-policy endorsements are retrievable in bulk; recursive wire-level decoding of every retrieved policy artifact found complete Python workload programs embedded (base64-encoded) in five artifacts of the sampled retrieved set (three distinct program texts)—including the Gboard training programs, each carrying its `_MAX_ZCDP` guard in the text (a constant compared at §5 against the budgets the program texts themselves state) (57 artifacts, §4.1): the embedded-program mechanism is confirmed by inspection of the retrieved set, as documented, and observed on currently served policies; the five are an existence count over the retrieved set, not a prevalence rate (§5); no differential-privacy budget, noise, or gating obligation is decided by the find. What survives is the residual-trust enumeration itself, and a checklist from a single-auditor case study.

This work is licensed under the Creative Commons Attribution 4.0 International License. To view a copy of this license visit https://creativecommons.org/licenses/by/4.0/ or send a letter to Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.

*Proceedings on Privacy Enhancing Technologies 2027(3), 1–24*

© 2027 Copyright held by the owner/author(s).

https://doi.org/XXXXXXX.XXXXXXX

# 1 Introduction

"Auditable" is a spectrum, not a switch. When a production machine-learning system claims its training pipeline is externally verifiable, the claim can mean anything from "we published a white paper" to "any outsider can re-derive every checkable assertion from public artifacts, today." For sealed-hardware *training* pipelines, it has not, to our knowledge, been mapped.

In October 2026, Google announced a federated learning system built on Trusted Execution Environments (AMD Secure Encrypted Virtualization–Secure Nested Paging (AMD SEV-SNP), Intel TDX—Intel Trust Domain Extensions) that moves model training server-side into TEEs [1]. Devices upload training data encrypted under keys held by a TEE-hosted Key Management Service (KMS); uploads are cryptographically bound to access policies naming the exact Python programs and container binaries permitted to process them; policies and binary endorsements are published to the public Rekor transparency log; and the KMS and data-processing binaries are claimed reproducibly buildable from open source. The system is in production, training Gboard English and Japanese next-word models.

The paper's verifiability claims name logs, keys, hashes, and build procedures—things an outsider can check. So we put the question in falsifiable form. Could an independent party, with no participating device, no Google credentials, and no insider contact, verify each claim from public artifacts, using documented procedures, as of a stated date? Where the answer is yes, we show the receipts: hashes, log indices, timestamps. Where it is no, we name the failing link—and what would fix it.

One clarification before the scores: *Verified* here means the public artifacts are mutually consistent—hashes match, log entries resolve, provenance chains close—not that the artifacts describe what actually runs on Google's servers. A malicious operator serving self-consistent artifacts—including in collusion with the log operator—defeats every outsider procedure; we score that residual: claims needing the further step to the running system score T, never V (§3).

**Contributions.** (1) A proposed seven-dimension audit checklist—induced from this deployment's own claim chain, exercised once on it, and not yet validated beyond this case study—for sealed-hardware training pipelines with a three-way scoring rubric (*verified* / *partially checkable* / *trust-required*, defined in §3). (2) A worked, artifact-only audit of Google's deployment—including the wire-format decode that found complete Python training programs embedded in five artifacts of the sampled retrieved set (three distinct program texts; an existence count, not a prevalence rate, §5)—grounded in artifacts pulled October 4–8, 2026, with later re-probes (§4). In our scoping search (§7)—a provisional sweep, not a systematic review—we found no earlier outsider verification attempt for a production sealed-hardware *training* pipeline. (3) A document-based juxtaposition of Apple Private Cloud Compute on the same dimensions—not a scoring. (4) A residual-trust enumeration: the set of claims that cannot be verified without Google's cooperation, which we argue *is* the audit's main deliverable.

**Scope and non-goals.** We audit *verifiability claims*, not the underlying ML or DP mathematics, and not TEE hardware security. We do not attack the system, access non-public data, or interact with Google; all procedures use public endpoints and are reproducible from our artifact package. One boundary runs through everything that follows: corpus and scores are a single auditor's—no second extraction, no human second scoring of the final matrix (§3)—so the corpus's recall is unmeasured.

# 2 Background: The System Under Audit

We summarize only what the audit needs; the system is described in Daly et al. [1] and a companion blog post [2].

*Architecture.* Client devices (here: Gboard, Google's keyboard app, so training examples derive from users' typing) encrypt training examples under a KMS-issued public key and upload them to storage, binding each upload to an *access policy*. The KMS is a cluster of TEEs ("sealed hardware" throughout means TEEs—hardware-isolated compute whose software measurements are cryptographically reported) running RAFT consensus over a rollback-protected key–value store. A server-side "root" TEE executes a workload author's Python program (signature `trusted_program(external_handle)`) and may delegate work to worker TEEs over a Noise-protocol channel, the root validating worker attestations (hardware-signed reports of the measurements a TEE runs). The KMS releases decryption keys only to TEEs whose attested measurements match the policy's *reference values* (four stack layers: Stage0 firmware digests + SEV-SNP security version number (SVN) thresholds; kernel/initramfs digests + boot cmdline; base OS image digests; container binary SHA-256).

*Verifiability design.* Before uploading, devices validate that (1) the KMS's TEE software and (2) the access policy are recorded in a public transparency log (Rekor [4]), including log-inclusion proofs. For Python workloads, the policy *embeds the program text* plus root/worker binary hashes, so "only that specific Python program, executed by those specific binaries, can access the uploaded data" [1]. Policies are published to Rekor; "external auditors can view how the program establishes DP guarantees" (e.g., a program-level `_MAX_ZCDP` guard, a zero-concentrated-DP bound). One qualifier: model architectures and preprocessing protos are sideloaded at runtime and deliberately *not* in the log, so the pinning covers the program, not everything it loads; the assurance that all privacy-relevant logic is pinned is itself trust-required (§5). The paper states the policy-listed binaries are "reproducibly buildable TEE containers built from OSS source code," and points to the `github.com/google-parfait/confidential-federated-compute` repository, whose docs map log entries to source commits [3].

*Threat model (ours).* We evaluate each claim's residual against six adversaries (labelled Adv. 1–6, to keep them distinct from the claim rows A1–G4): **Adv. 1** honest-but-curious pipeline operator; **Adv. 2** malicious (active) operator; **Adv. 3** holder of a compromised endorsement key; **Adv. 4** TEE hardware/firmware attacker; **Adv. 5** malicious workload author; **Adv. 6** network observer. Capabilities: Adv. 3 holds one endorsement key; Adv. 4 operates beneath the software measurements (hardware/firmware); Adv. 5 authors workloads, not infrastructure; Adv. 6 observes network traffic only. Adv. 4 and Adv. 6 are not scored claim-by-claim: no procedure in our persona engages them, so they are carried as residuals (Table 6)—a scope boundary, not a failed check. The operator is *not* assumed honest. We treat operator↔log-operator collusion as an explicit

*assumption* of the threat model, not as a finding: Rekor is operated by Sigstore/OpenSSF, and Google co-created Sigstore (with Red Hat's Open Source Security Team) [22]—a founding association that establishes neither current operational control nor shared signing authority, and we allege neither. The operative question is what a colluding log could hide and what would detect it (§7).

## 3 The Audit Framework

Our framework has three parts.

**Dimensions.** Seven, spanning the chain from silicon to operator: (1) *attestation binding* (device→KMS→TEE chain of trust); (2) *code transparency* (source availability, reproducible builds, provenance); (3) *policy expressiveness* (what the access policy pins down, and whether the "complete set of workloads" is enumerable); (4) *key lifecycle* (endorsement-key publication, rotation, custody); (5) *firmware/boot chain* (reference values, hardware roots); (6) *side-channel surface*; (7) *operator-insider residual* (what a privileged insider could do or hide). Coverage is of claims an outsider can state a procedure for: one obligation the guarantee composes from—DP output gating with atomic release—has no checkable claim or artifact in the deployment, hence no row; Table 8 marks it not examinable. One further boundary: a conjunct asserting correspondence between public artifacts and the vendor's production execution is not rowed—its checkable content is exhausted by the rebuild row (B2), and the remainder (that the production run used this source and recipe) is unobservable in principle, carried as deployment residual (T) at its host row (B1, Table 9). The splitting rule below operates where a source asserts the separated part as a claim in its own right (row A2b), not on a subordinate clause of a publicity sentence.

**Scoring.** Each checkable claim is scored **V** (verified: we executed the check end-to-end on public artifacts), **P** (partially checkable: sound in principle, but degraded, blocked, unexecuted, or inspection-only in current practice—the basis states which), or **T** (trust-required: no outsider procedure exists that could decide the claim at its full scope at feasible cost). Scores are evidence-anchored: every V cites the artifact, hash, and date; every P names the broken link; every T names the trusted party. Terminology, used consistently below: *verified* means executed and recomputed by us; *authenticated* means a signature check passed; *observed* means returned by a public endpoint and recorded, without independent recomputation. Four evidence modes are kept distinct: *human scoring* (Table 10), *automated rescoring* (Appendix A), *reimplementation*, *independent reproduction* (§4). One vocabulary discipline governs every count in this paper: a *row* carries one score letter, a *table* aggregates by counting rows carrying a given letter, and a distribution count printed below is a count of rows—never of artifacts, payloads, policies, or endorsements, which are counted only in §4, always with their population named. Four rules keep the rubric honest. First, disclosure: a row scores V on a vendor statement only when the claim audited *is* the disclosure (e.g. that a limitation is acknowledged); where the claim is a fact and the only support is the vendor's statement of it, the row scores T—a verified statement does not verify a fact. A companion precedent: where the claim audited is that an artifact is public (source, build recipe, reference values), fetching and inspecting the artifact *is* the execution, and the row can score V (row B1; the V covers the publicity conjuncts only—the built-from conjunct of B1's wording is scored at row B2). Second, scores grade *checkability*, not truth: where an executed check contradicted the claim as worded, the row's basis states the contradiction, and that P score should be read as a negative finding, not reassurance. The marker for that case is *, and no row of the matrix carries it: row C5's earlier marker was withdrawn at this revision (§4.5)—read as a whole, the vendor passage states an endeavour and a one-year floor, and the lapse occurred past that floor—so C5 scores P as an executed observation, not a contradiction. Two components are reported for every row and must not be read as one: the letter positions the row on the checkability spectrum—three grades, each refined by the execution subfield the basis states (executed end-to-end, executed with qualification, inspection-only, unexecuted, or no procedure); and the outcome of a decided claim is carried separately, by the decisions count of §6 (and, for a contradicted claim, by the * marker). Third, a row whose procedure exists but was not executed is scored P and flagged "unexecuted" in its basis; it is never scored V (nothing was executed) or T (a procedure exists). One qualification (row C3c): a procedure executable only over a bounded window that cannot decide the claim at full scope does not trigger this rule—the row scores T, its basis reporting the bounded execution and the costing (§4.4). The letter T therefore rests on three distinct grounds, kept distinct in the bases: no procedure exists; a procedure exists but is not open to the scored persona; and a procedure exists and is open but cannot decide the claim at full scope at feasible cost on the routes and resource assumptions stated for the row (row C3c). The rule governs claim rows; a scope-limit record, which scores no vendor claim, is flagged on its own terms (row G4, Table 1). Fourth, a line for code inspection, which several rows turn on: a procedure counts toward P only if the scored persona could execute it on public artifacts *and* its execution would decide the claim as worded. Static inspection of the public client code path meets that test for the published client implementation (row A2), and we executed it (2026-11-04): the published client library's attestation verifier checks the upload-key endorsement against the KMS verifying keys and the pipeline configuration against the policy verifying keys, verifies Rekor inclusion proofs by root-hash recomputation, and cross-checks the policy digest carried in the key endorsement. What that decides is the public client implementation: A2 is thus scored as a source-property claim—demonstrating the behaviour, by running the verifier against a live endorsement, is the execution a V would require, and it was not performed, so inspection, which decides the code as written, carries the P. "Decides", here and for rows C1/C2 below, settles the published-artifact proposition; it is not a decision in the sense of §6's count, which comprises the claims an executed procedure decided on the claim as worded against live artifacts—returning *yes* (the V rows) or *no* (a P* row); no P* row exists at this revision, so the count is the V rows alone—and A2 and C1/C2 score P because that V-grade execution was not performed. The execution the claim requires is running the verifier itself; a clean-room reimplementation would be a stronger, independent variant, not the requirement—what either adds over inspection is the behaviour of the code as run (its parsing and I/O paths), not a different proposition. That production devices run it is not observable, and it scores separately as row A2b (T)—the corpus rule below, applied to A2 at this revision on the same footing as the enforcement

halves of C3 and C4 (rows C3b, C4b): bundled statements are split wherever their parts are distinct propositions resting on different evidence and decided by different checks. Rows C1 and C2 rest on the same line from the other side: inspecting the published protocol definition and the guides' worked policy decides the pinning and budget claims as worded for the published artifacts, so they score P by inspection although no live policy byte was executed. They are the weakest P scores in the matrix: inspection of published artifacts is the evidence class that failed on row B4, where the decisive content stayed invisible until wire-level decoding superseded the scan (§4). We keep the line because the claims are about the published artifacts themselves, and B4's failure was of the scan applied to retrieved bytes, not of inspection: the same bytes, wire-decoded, decided B4. Executing the pinning and budget fields of live served policies would score a different proposition—what served policies in fact pin and carry—which no extracted claim asserts. It does not meet the test for claims about runtime behaviour inside the operator's TEEs (rows A3, C4b, D3, G3): inspecting server code shows what the code says, not that the running system does it, and the running binary's identity is itself reachable only conditionally, by a participating device under row E2's hardware assumptions (§5); those rows score T, although their code is equally public.

The acceptance line across the four rules is uniform. A row is decided—and can score V—only when a procedure the scored persona can execute on public artifacts decides the claim as worded against live artifacts (a hash, signature, inclusion proof, or build recomputed; for a publicity claim, the artifact fetched from its live source—rows A1, B1). Inspection—reading artifact content—decides a claim only as a statement about the artifacts read, and carries P (rows A2, C1, C2, B4). An executed procedure that decides the claim only in part, or under a stated qualification, scores P as executed-qualified (rows B2, B3, C3, C5, D1); a procedure that exists but was not executed scores P, flagged unexecuted; and where no open procedure can decide the claim at full scope at feasible cost, the row scores T, on the ground its basis names. The full 29-row matrix is in Appendix B. We know of no outsider-audit rubric for sealed-hardware *training* pipelines to benchmark against (existing TEE guidance targets hardware assurance, not claim-by-claim checkability); the seven dimensions are our proposal, derived from the system's own claim chain.

**Claim corpus and extraction.** The scored claims are extracted from a fixed corpus: Daly et al. [1] (v4, 2026-09-30), the companion blog post [2], and the repository documentation as fetched 2026-10-08 (the endorsement and attestation inspection guides, the KMS design document, and the reference-values files). A statement enters the matrix iff it asserts a property an outsider is invited to check from public artifacts; bundled statements are split wherever their parts are distinct propositions resting on different evidence and decided by different checks (the C3/C3b, C4/C4b, C3/C3c, and A2/A2b splits are instances of this rule, not exceptions—each was recognized where the parts' checkability diverged). A bundle is retained as one row where every part is undecidable for the scored persona on a shared ground—no outsider-executable procedure reaches any part—so no divergence is masked; the parts are then named in the row's basis and in §5 (row D3: neither the binding's in-TEE enforcement nor the TTL's elapsed-time assurance is reachable by an outsider procedure). Each row is assigned to the dimension naming the chain layer at which its check executes. The verbatim claim texts and per-row evidence are in the artifact's `AUDIT_MATRIX.md`; Appendix B prints the matrix, the claim wordings as scored (Table 9), and the second-scorer agreement table (Table 10). Rows created by the split rule are audit-derived decompositions of single vendor statements, not separate vendor assertions; their split history is recorded per row in the artifact. Row totals count these components, and §5 reports the distribution's sensitivity to the decomposition.

**Verifier classes.** "Outsider" is not one vantage point. We distinguish (i) the unauthenticated public observer—no device, no credentials, no operator contact; this is the persona our scores are reported for; (ii) the independent researcher with a commodity cloud account (bulk-data routes and rebuild-class compute; used for the §4.3 rebuild); (iii) the participating device owner, who can additionally validate attestations device-side; and (iv) the cooperating external auditor, whose vendor-facilitated access places them outside "outsider" as we use the term. Where a row's answer changes by class, the basis says so; a T score means no procedure for class (i), not that no class could decide it.

| Class | Vantage | Distinctive access |
|---|---|---|
| (i) | Unauthenticated public observer | No device, no credentials, no operator contact; public artifacts only |
| (ii) | Independent researcher, commodity cloud account | Bulk-data routes (the BigQuery dataset); rebuild-class compute |
| (iii) | Participating device owner | Additionally validates attestations device-side |
| (iv) | Cooperating external auditor | Vendor-facilitated access; outside "outsider" as used here |

The framework is a proposal, not a validated instrument: the original scoring is by a single auditor; a second-scorer plausibility check on the condensed rows and an automated artifact-level re-scoring of the matrix as executed are reported below; no test–retest was done. A limited reliability check on the *scores* (2026-11-04): a second scorer, blind to the first scoring, re-scored all 23 condensed rows as then constituted (the added rows were not re-scored) and agreed on 21 of 23. Both disagreements argued our scores were too generous, and both were adopted at the time; on the corrected record enumeration stands at P, for the reasons §4.4 states (the scorer judged the evidence as presented, which contained the first pass's mistaken premise about the key index). Chance-corrected agreement is Cohen's $\kappa = 0.85$ ($n = 23$; the 3×3 table and both itemized disagreements are in the artifact's second-scorer record). $\kappa$ describes agreement with the scores as held at the time—one (enumeration) has since changed on corrected evidence, so $\kappa$ does not characterize the published matrix. The matrices have gained rows (the C3 and C4 splits, §4.4 and §5, and at this revision the A2 and C3c splits—both outside both exercises); the exercise's counts describe the matrices as then constituted. One limit stands: the second scorer worked from the evidence *presented in this paper*, not from the raw artifacts, so it tests whether our presentation supports our scores—a plausibility check on a superseded constitution, and the only human second-scoring in our record.

On 2026-11-04 an automated scorer—a separate agent in the same pipeline, blind to the published scores, not a human auditor—re-derived all 27 rows of the audit matrix as executed—the matrix in the artifact's `AUDIT_MATRIX.md` (the published full matrix without the revision-stage split rows A2b, C3c)—from the artifact package

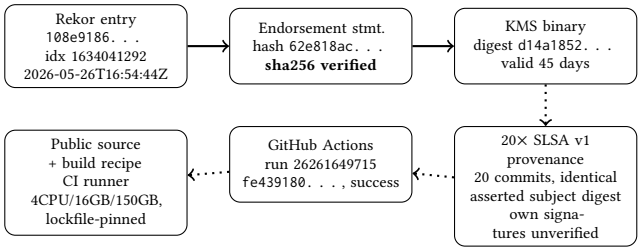



**Figure 1: The outsider-verifiable chain for the KMS binary. Solid edges were re-derived by us from public endpoints for the guide's documented exemplar (hash recomputation; the endorsement statement's own signature verified, §4.1); the from-scratch binary rebuild was executed with the project's own CI invocation (§4.3). Dotted edges rest on provenance statements fetched from the GitHub Attestation API whose own signatures we did not independently verify (§4.1).**

under the rubric above. It agreed on all 27 rows and changed none—its raw recorded agreement, before the adjudications that follow. Four rows where a competent scorer could land elsewhere (A3, A4, B2, G4) were adjudicated on the rubric's own precedents; the score stood in each (adjudications in the artifact's re-scoring record). We report the raw outcome only, with no chance-corrected statistic: the scorer shares our pipeline and rubric, so the exercise evidences only that the as-executed matrix is *reproducible from its own artifact-level evidence* under its own rubric—not independent confirmation, and not the original scoring act's reliability. In short, it is a rubric sanity check—the rubric re-applied to the artifact-level evidence by the AI tool disclosed in the AI Use statement, run as a separate agent. No human second scoring of the final matrix from the raw artifacts exists: every score is a single auditor's, and the audit is a single-auditor case study throughout. All quantitative claims are single measurements or census counts over the audited artifacts; we make no inferential comparisons, and rates derived from single measurements are reported as order-of-magnitude where the text says so.

**Comparator design.** A single-case audit should indicate whether its dimensions separate design choices from verification gaps or merely relabel one system's properties. Our only instrument for that here is a juxtaposition: a second production confidential-computing deployment—Apple Private Cloud Compute (PCC)—set against the same dimensions, from documentation (Appendix D). PCC is inference-only and architecturally different. The juxtaposition is illustrative, not a validation: document-based and unscored, it can show only that the dimensions return a different mixture of design choice and verification gap for a second system on paper, not that they discriminate in general; whether they do is part of what other audits must test.

## 4 Worked Audit

The October procedures below were executed October 4–8, 2026, from an unprivileged vantage point (no device, no credentials): verifier class (i) of §3. The single exception is the §4.3 rebuild (2026-11-04), which used class (ii), a commodity cloud account, as flagged there; the further 2026-11-04 additions are listed next. Scripts and outputs are in the artifact package; every hash below was recomputed. **Audit timeline and amendments.** Re-probes followed on 2026-10-15 (inclusion-proof verification, key-index endpoint), and an endpoint re-probe on 2026-11-05 (Appendix C). On 2026-11-04, under amendments frozen before any of their runs, we added (we flag the pattern ourselves: both amendments raised scores, and they were written after the October results were known—frozen before their runs, but not blind to the audit's direction; readers should weight the two raised scores accordingly): the signing-key and KMS digest censuses, an independent re-implementation of the inclusion-proof verification, the provenance-commit classification, and the measured retrieval-rate experiment (§4.1, §4.4); a second amendment the same day added the all-keys census, the exemplar's signature authentication (checkpoint, signed timestamp, endorsement signature), and the policy-payload classification (§4.1). Two review-stage additions later the same day—a wire-format decode of every retrieved policy artifact (superseding the string-scan classification for row B4, §4.1) and the executed static inspection behind row A2 (§3)—are flagged where they occur. Confirmatory status: the amendment experiments were frozen before their runs and are confirmatory for the procedures they specify; the §4.3 build success is not—the frozen command failed at the era tree and the working invocation was found in the project's CI workflow mid-audit, so that result is an exploratory rescue, reported as such in §4.3. One deviation is disclosed: the signing-key census was extended from the frozen first-10 sample to all 115 returned entries; both results are reported. The frozen amendment text is in the artifact package (`AMENDMENT-2026-11-04.md`); the experiments changed two letter scores (A1 to V; C3 to P) and the stated basis of two further rows (B2, B4); the review-stage additions changed no letter score; every other October conclusion stands.

### 4.1 Log → endorsement → binary → source

Figure 1 shows the chain we verified end-to-end; we recomputed the edges from the public endpoints, not copied from documentation—retrieval and hash recomputation on 2026-10-04/08, inclusion-proof verification on 2026-10-15, signature authentication on 2026-11-04—and executed the rebuild on 2026-11-04 (§4.3). Terms, at first use: *hash-verified* means we recomputed the SHA-256 and it matched; a *live* artifact is one the store still served on the audit date; the *wire format* is an artifact's serialized protobuf field stream, readable field by field without the vendor's schema; a *complete program* is embedded text that parses as a Python program—imports, definitions, and a training entry point—not a fragment or a marker string.

We walked the documented verification chain for the KMS binary. Google's audit guide [3] gives a worked example: Rekor entry UUID `108e9186. . .` (full UUID in Appendix C). We fetched it from `rekor.sigstore.dev/api/v1/log/entries/{uuid}`: log index **1634041292**, integrated time **2026-05-26T16:54:44Z**. The entry body names endorsed hash `62e818ac. . .`. We fetched `https://federatedcompute-pa.googleapis.com/data/transparency/sha2-256:62e818ac. . .` and computed SHA-256 over the returned bytes: **it matches the endorsed hash exactly**. The payload is an in-toto (a supply-chain attestation format)/Oak endorsement statement whose subject is the endorsed KMS artifact digest (the application bundle; §4.3) `d14a1852. . .`, valid 2026-05-26 through

2026-07-10 (a 45-day difference—46 calendar days inclusive; day counts in this paper are differences). Three notions must not be conflated here: *live validity* (after 45 days the endorsement authorizes nothing new), *historical verifiability* (the signed statement remains checkable while its bytes and keys are served—we verified this expired exemplar's signatures on 2026-11-04, months after expiry), and *availability* (whether the store still serves the bytes at all—the binding constraint, §4.5).

Provenance: querying the GitHub Attestation API for that digest returned **20 SLSA v1 provenance statements from 20 distinct source commits** (spanning 2026-05-22 to 2026-06-03), every one with subject digest equal to `d14a1852. . .`, builder `github.com/google-parfait/confidential-federated-compute/.github/workflows/reusable_build.yaml` (the statements' own signatures were not independently verified). We did not validate each statement's Sigstore bundle (its DSSE signature, Fulcio certificate chain, and Rekor entry): the provenance observation scored here is the statements' published existence and subject-digest agreement; their authenticity is carried as residual trust (row B3). The pattern is consistent with toolchain determinism, though also with a stale build cache that never rebuilds. Whether the 20 commits even touched the KMS build inputs is answered by classifying every commit's changed files through the GitHub API (2026-11-04): only **1 of 20** commits modifies files under `kms/`; 5 more touch the root `MODULE.bazel` (shared dependency declarations); **14 of 20 touch neither**. For those 14, an identical KMS digest is expected on the screened paths whether or not the toolchain is input-sensitive, so the 20/20 observation limits what can be claimed about toolchain determinism rather than proving input-sensitive reproducibility; and we did not compute the commits' full dependency closure, so we claim only that the identical digest is unsurprising for those 14 on the screened inputs, not that no build input changed. The referenced Actions run (26261649715, "Build and attest," conclusion `success`, head SHA `fe439180. . .`) and the full build recipe are public; the recipe pins its dependencies to a lockfile (`--lockfile_mode=error`—pinning, not network-hermeticity: our first build attempt failed at dependency download, §4.3), runs on a 4-CPU/16GB/150GB runner, and signs provenance via `actions/attest-build-provenance@v1` to Rekor.

We state the trust assumption: these 20 statements are issued by Google's own CI infrastructure, so they are evidence against a buggy or non-reproducible toolchain, not against a malicious operator who controls that infrastructure (threat model Adv. 2); the Adv. 2 residual stands, scored accordingly in §5. A second scoping note, on the inclusion proof: on 2026-10-15 we executed full RFC-6962 inclusion-proof verification—recomputing the leaf hash from the entry body (it equals the UUID's suffix; the UUID's prefix is the tree ID, 1193050959916656506) and folding the API-returned sibling hashes to the checkpoint root with an implementation validated on synthetic trees. The proof verifies at leaf index 1,512,137,030, while the entry's own `logIndex` field reads 1634041292: different index spaces, not a contradiction—Rekor shards its log, the `logIndex` is the *virtual* index (the leaf index plus preceding shards' tree lengths, 121,904,262 leaves here), and the proof is per-shard, against the entry's own tree checkpoint. The binding values: leaf hash `4e8e6888. . .` (RFC 6962); signed checkpoint (origin `rekor.sigstore.dev` – 1193050959916656506): tree size 2,958,662,216, root `f79c1235. . .` — the proof's size and root; 1,512,137,030 + 121,904,262 = 1,634,041,292. We record a correction: our first reading treated this pair as one index space and reported an API self-contradiction; Rekor's `VirtualLogIndex` semantics dissolve it, and a second, independent RFC 6962 implementation (2026-11-04, fresh entry and proof) confirms the fold. Log membership for the exemplar is therefore verified, and on 2026-11-04 we closed the remaining authentication gaps: the checkpoint's signature verifies against Rekor's public key (log ID `c0d23d6a. . .`), the entry's signed timestamp verifies over Rekor's canonical entry JSON (a negative control: the same fields serialized in a non-canonical order do not verify, so the check discriminates the serialization, not the presence of a signature), and the endorsement statement's own signature verifies under the endorsement key embedded in the entry—which is identifiably the published application endorsement key, fingerprint `ea0d1f8f. . .` (§4.2). One integrity check was not run: cross-checkpoint consistency proofs (append-only consistency between signed checkpoints over time) were not verified; the log-integrity scores rest on inclusion against the signed checkpoint and the SET as above, and the collusion residual of §5 is carried on that basis. A final scoping note: the chain replicates the guide's worked example, so it verifies the published procedure on the vendor's chosen entry, and it verifies *logging*—that this entry, endorsement, and digest were recorded and authenticate as shown—not deployment.

**Beyond the worked example: the signing-key census (2026-11-04).** Under a frozen amendment, we ran a census of every Rekor entry sharing the exemplar's entry-signing key, retrieved through the index endpoint of §4.4—115 entries in total. No entry was chosen by us; the population is nonetheless one key's entries, reached through the vendor's worked example; the all-keys census widens the frame. For each we fetched the endorsed payload from the transparency store and recomputed its SHA-256. **110 of 115 payloads byte-match their endorsed hashes** (48 of them KMS endorsements, the rest ledger and other Oak endorsements); **zero mismatch**; the 5 failures are the earliest entries, whose payloads the store no longer serves (HTTP 400). What this verifies is the entry→payload hash link on samples beyond the exemplar—plus, for the 48 KMS endorsements, the payload fields themselves: every one's subject digest and validity window parses, and the 48 carry **24 distinct endorsed KMS artifact digests** (2025-06-04 to 2026-11-03), so the endorsed digest demonstrably changes as the system evolves; the 20/20 identical-digest window above is a 12-day slice of a moving target (the executed rebuild of §4.3 covers one of those twenty commits' trees). That entry→payload link verifies on those samples wherever the store still serves the payload—and the 5 unserved payloads are themselves an instance of the retention finding (§4.5).

**All-keys census (2026-11-04).** We then extended the census to every published endorsement key we could reconstruct from the repository's reference values and the guide: 19 keys (6 KMS-layer keys, the application key above, 8 Gboard policy keys, and 4 other apps' policy keys), returning **509,569 entries** in total through the same index endpoint, no credentials required. The counts are the index's returns, whose completeness against the log's tree we did not independently verify (row C3c, §4.4). (A seeded cross-check

followed in the review stage: 24 entries, six per key for four keys—every inclusion proof folding to its served checkpoint root, every checkpoint signature and signed entry timestamp verified; §5.) Payload checks were exhaustive for every key with at most 120 entries—all KMS-layer keys, the application key, and Gboard policy key 2 (Gboard policy keys 3–8 returned zero entries)—and sampled for the five keys with more (Gboard key 1, Google App, Pixel Search, Private Compute Services, Recorder)—30 spaced entries per key, 31 for Gboard policy key 1 (the 30 plus the guide's named entry); the per-key table is Appendix Table 3. Of 428 payload fetches attempted in this extension, **320 byte-match, zero mismatch**, and 108 returned HTTP 400 (the exemplar key's 115 entries were fetched in the signing-key census above; including them, 543 fetches, 430 matched, 113 unserved—concentrated, not uniform: 96 of them under the five KMS boot-chain keys, one unserved in 151 attempts across the five sampled app-policy keys, and two small keys (KMS container key 1; Gboard policy 2) wholly unserved); the unserved payloads whose age position is visible sit at the old end of the record—the exemplar key's five earliest entries; the guide's 2025 worked example has lapsed (§4.5). Two fingerprints printed in the guide (kernel `6052f352...`, initramfs `5f884b69...`) appear in no current reference-values file—the guide's own example entries for those layers embed the current keys—so the published key inventory shows drift, a point §4.4 returns to. The app policy keys account for 509,185 of the entries, and their payloads settle the policy question of §5: every retrieved policy endorsement is an Oak statement with subject "policy" pointing to a binary-protobuf policy artifact (21 second-level artifacts fetched—11 policy artifacts: 3 Gboard and 2 per other app key; plus 10 KMS-layer statement digests as controls—all hash-matched). Classification method: JSON statements were parsed field by field, and every retrieved payload and all 21 second-level artifacts were string-scanned for `_MAX_ZCDP`, `ZCDP`, and program markers (`def`, `import`, `python`, `program`). A string scan cannot see encoded content, so we then subjected the policy artifacts to a recursive wire-format decode (2026-11-04; decoder and record in the artifact, `scripts/e10_wire_decode.py` and `records/E10-E11-results.md`): a fresh retrieval on the census's sampling (derivation below) served the payloads referencing 57 distinct artifacts; every artifact was fetched, hash-matched, and decoded field by field at every nesting depth, with every long encoded field additionally scanned after base64 and hex decoding and a compression-magic scan throughout. The retrieval behind the decode is a fresh draw, distinct from the census classification sample above: re-running the sampling over the six app policy keys (exhaustive at ≤120 entries, else 30 evenly spaced, plus the guide's named Gboard entry) attempted 159 payload fetches, of which 149 were served and hash-verified (8 returned HTTP 400 and 2 entry fetches failed; per-key counts are in the artifact record), referencing the 57 artifacts just named. The 57 are not one kind of object: a policy artifact either wraps a workload—carrying the program itself, base64-encoded, as an embedded field, the mechanism the system paper documents—or is a configuration proto, pinning digests, budgets, and parameters, with no program in any field. The decode's classification is by that content—a complete program in the embedded field, or the bundle configuration with no program text in any field, per-field or decoded—so the count that follows is an existence count across both forms, not a share of the form that can carry a program (the denominator caveat of §5). The six keys are the app policy keys of Table 3; their per-key composition is Table 4 (and the artifact record). **The decode found complete Python workload programs embedded, base64-encoded, in five of the 57 artifacts—three distinct program texts**: a Gboard ACT training program (44,189 B decoded) and a Gboard HMM SentencePiece training program (32,314 B), each carried by two artifacts, and an AGSA/OmniEnt differential-privacy synthesis program (10,676 B) endorsed under the Google App key by current statements—each carrying its `_MAX_ZCDP` guard in code (a constant compared at §5 against the budgets the program texts themselves state). The remaining 52 artifacts contain no program text in any scanned form—plain, nested, or encoded—and are binary-protobuf configurations; no first-level payload contains a program in any scanned form. The embedded-program mechanism is therefore verified as documented *and* observed on currently served policies, the Gboard training workloads among them; the October string scan's non-detection is superseded, and we flag the correction. The observation's scope is the retrieved set (exhaustive for the small keys, sampled for the five large ones); program-carrying artifacts are the minority form (five of 57; an existence count, §5). The decode excludes program text in the forms above, not content encrypted under a key we do not hold (no field carries compression/encryption markers—reported as observed, not as proof about opaque content). Three denominators should not be conflated: endorsement-statement retrieval attempts (543 across both censuses—430 served, hash-matched, classified; 113 unserved); the distinct policy artifacts the decode sample's statements point to (57, all fetched and hash-matched); and second-level artifacts fetched in the census classification (21: 11 policies, 10 KMS-layer controls).

## 4.2 Endorsement-key inventory

We parsed `access_policy_reference_values/gboard.txtpb` and recomputed each key's SHA-256 fingerprint from the raw DER: **8 unique ECDSA-P256 endorsement keys plus 1 Rekor log-verification key** for Gboard (the guide's 2025 example showed one; that example's key class is unverified, §5). The KMS-layer endorsement keys are published in `kms/reference_values` (stage0, kernel, initramfs, kernel cmdline, system-image, and two container-layer keys—one of them the application key, counted separately in the §4.1 frame). The keys are not anonymous indices: the key embedded in the exemplar entry of §4.1 decodes to fingerprint `ea0d1f8f fed9512019a2ec968790263d88ea3324c6a6a782114e5ea1be4fd38f`, exactly the guide's *application* endorsement key—so the §4.1 census by that key is an endorsement-key census (the identification §4.4's analysis rests on). The published key inventory is observable; the key row scores P, because rotation is only inferred—the guide's example shows one key, not shown to be the exhaustive 2025 set, and its key class is unverified (§5). The reference values confirm the freshness policy: endorsements accepted only if integrated after 2025-02-15 and within 90 days of verification (`signed_timestamp` constraints); this window is distinct from the endorsement's 45-day validity above.

## 4.3 Reproducible-build attempt

The strongest checkable claim is that policy-listed binaries are reproducibly buildable from OSS source. We attempted the build our frozen protocol specified: `bazelisk build -c opt //kms:main` (Bazel 8.2.1, repo-pinned) on a 2-vCPU/7GB-RAM VM. The build failed *before compilation*, at dependency download: Bazel's Java HTTP client throws `NoSuchElementException` in `doTunneling` against our environment's authenticated egress proxy (exit 37; log in artifacts), recorded as **attempted, environment-blocked**—not as evidence against Google's claim. Two further attempts on 2026-11-04 ran the same command on a documented-class cloud environment (4 vCPU/30 GB, unproxied) and failed at target resolution, before any compilation: the first at current HEAD, where the `kms` package has been deleted upstream; the second at the pinned exemplar-era commit (`fe439180. . .`), where the root `.bazelignore` excludes the `kms` directory, so the command does not resolve there either. Neither executed the documented build; neither is a result on the claim. The repository's "Build and attest" workflow—the workflow in this binary's provenance chain—documents the invocation: the `kms` directory is a standalone module root, built from inside itself via `bazelisk run -c opt //:install_release_binaries`, which installs the application bundle as `kms/container.tar`. A third cloud attempt on 2026-11-04 ran that invocation at the pinned exemplar-era commit on a hosted environment (Kaggle kernel, ≈36 min, 3,813 actions, exit 0; the host publishes no resource class—a gap in our run record that we flag rather than fill): **the build completed (exit 0), and the SHA-256 of the produced `container.tar` equals the endorsed KMS digest `d14a1852. . .` exactly**. The documented-build claim is therefore verified by outsider execution for the exemplar-era tree—via the project's CI invocation. A second clean run of that invocation on 2026-11-10—a fresh clone at the same commit, run from inside `kms/`—reproduced the endorsed digest bit-for-bit again; the run's host self-reported Ubuntu 24.04.5, 4 vCPUs (Xeon @ 2.20 GHz), ≈32 GB RAM, and Bazel 8.2.1, and publishes no resource class. One precision: the workflow file is the only documentation of the working invocation—the prose docs and the audit guide do not state it; following the prose docs alone leads to the plain command, which does not resolve at the era tree. We record the drift as an operational finding: from outside, on the prose route alone, a documented command that does not resolve presents as a failed reproducibility claim, even when the source and tools are fully public. We count the result as verification by the project's documented-in-CI procedure and, as §4 states, as an exploratory rescue. By *independent reproduction* we mean a from-scratch build of the same source tree by a party other than the vendor's CI, compared bit-for-bit against the endorsed digest: this execution is one; the 20 provenance statements of §4.1 are the vendor's CI asserting the same digest, not independent reproductions. Three caveats: (i) the executions are at one commit—the exemplar's—so bit-identical reproducibility is established for that tree, while input-sensitivity across the 20 provenance commits of §4.1 remains observational (identical digests, most from commits that did not touch the screened build inputs); (ii) the bundle form matters: the endorsed digest is over the container bundle that invocation produces, which is the artifact the endorsement names; (iii) the run record does not pin the hosted environment's resource class—a record gap only: the digest comparison itself is environment-independent. Nothing in the procedure is specific to the exemplar commit—the invocation, module layout, and toolchain pin (`kms/.bazelversion`: Bazel 8.2.1) are the era tree's—so the same route could replicate each other provenance commit; but success and the ≈36 min runtime are demonstrated for the exemplar only.

## 4.4 Log scale and the enumeration problem

Binary-searching Rekor's index-retrieval endpoint, we measured the public log at ≈**3B** entries on 2026-10-04, the aggregate across shards in the virtual index space of §4.1—an order-of-magnitude figure: both measurements came through that endpoint, and every quantity derived from it in this section (rates, window size, scan durations) inherits that status; the growth-derived quantities inherit the May baseline's proxy character—an inference from the worked-example entry's index, not a measurement of the log—growing ≈**1.4B** entries over the 131 days since 2026-05-26: ≈**10.7M entries/day** (≈124/sec, both order-of-magnitude; our 2026-10-08 re-measurement implies a higher short-window rate—consistent with ingestion spikes in the Sigstore log's bursty traffic; we ran no diagnostic that separates that explanation from a measurement discrepancy—so the 131-day average is the long-window figure). (Methodological note: on 2026-10-05 the `logIndexes` endpoint returned empty results for all queries, including previously valid indices; we did not diagnose the cause and draw no conclusion from it. The two measurements agree on scale.) This measurement matters because the paper claims third-party auditors "can know the full set of potentially active binaries and workloads" from the log [1]. Enumerating that set requires finding entries *by endorsement key*—and here our first pass erred, changing a score. Rekor's key index is keyed by the public key that signed each log entry; assuming endorsement keys sit only inside entry payloads, we tried the entry-retrieval endpoint with an endorsement-key PEM, observing timeouts. Both were the wrong instrument. The index endpoint `POST /api/v1/index/retrieve` accepts a public key and returns its entries: queried on 2026-10-15 with the key embedded in the exemplar entry, it returned 106 entries (115 on 2026-11-04; the census of §4.1 uses the later count). And that embedded key is identifiably a *published endorsement key* (§4.2): the guide prints its fingerprint, finds this entry by searching that fingerprint with `rekor-monitor`, and queries the Rekor BigQuery dataset by `KeyDigest`. Endorsement entries—for binaries and for policies alike—are therefore discoverable by published endorsement key through three routes: the index endpoint (no credentials—the route our score rests on), `rekor-monitor` fingerprint scans, and the BigQuery dataset (which requires a GCP account). The all-keys census of §4.1 is the executed form of that discovery: 19 reconstructable published keys → 509,569 entries, payload-verified as described there. We therefore score the enumeration claim in three parts. What the *published* keys have endorsed is enumerable, and we executed it: **P** (row C3)—not V. The letter grades entry discovery alone, and discovery falls short on one ground: (i) Two fingerprints printed in the guide could not be queried (no key material, §4.1; no fingerprint-directed log search was run for them), so even the published set is not enumerable exactly as published. Two further bounds attach to the census's other observations, not to the discovery letter (§5 states the same

division): (ii) Enumeration finds *entries*; reading what they endorse requires the payloads, and payload retention lapses (§4.5): an enumerated historical endorsement may no longer be retrievable. (iii) For the largest policy keys our payload classification is sampled, though entry counts are exact. The relevant inventory is not every key that might exist but the *acceptance set*: the reference values against which client software validates endorsements before trusting an upload. That set is public (our census frame); a key outside it would not be accepted by conforming clients, and the residual inside it is a published fingerprint with no reconstructable key material (two, §4.1). The claim—that the acceptance set as deployed on devices is exactly the published one, i.e. nothing was endorsed under an unpublished key that devices would accept—scores **T** (row C3b): no outsider procedure can reach it, and the inventory is vendor-asserted and shows drift (two guide fingerprints match no current reference-values key, §4.1; keys published anywhere else would be missed by construction). A third part, separated at this revision: that the index endpoint's *returns* are complete against the log's tree—omits no entry for a queried key—scores **T** (row C3c). The seeded cross-check of §4.1 authenticates returned entries, but a sample drawn from the returns cannot show the endpoint omits nothing. One bounded cross-route reconciliation was executed under a 2026-11-05 amendment: over 1,432 log indexes, the log-entries route found the index route's 61 entries of one policy key, every index accounted—no omission in that window (the Google App policy key; the amendment's window—1,432 contiguous indexes holding 61 of the key's entries, above its 30-entry floor; the remaining 1,371 bodies are other keys' entries); outside it, and for the BigQuery and `rekor-monitor` routes, no count check was run. Row C3c's T rests on scale, not on the absence of a procedure in principle: the only route we evaluated that decides completeness for a key is a full-log scan collecting that key's entries. The costing above measures that scan over the freshness window (≈3–6 years); C3c's extent is the whole tree (≈3B entries), costing ≈10–19 years at the same rates (both scale linearly in tree size and inversely in the sustained rate). Our feasibility reading is operative: a continuous multi-year scan at the best measured credential-free rate is outside any plausible audit budget for persona (i). The T is therefore conditional on the evaluated credential-free strategies (the measured batched-sequential implementation; `rekor-monitor`-manner scans over the same paths); we claim no lower bound over every conceivable credential-free method. The scan has been executed only over the bounded window above. Row C3's proposition is discovery: the entry set endorsed under each published key, as returned by the index endpoint. Row C3c's claim has the log's whole tree as its extent and omission-freedom as its criterion: every tree entry signed under the key appears in the endpoint's returns. The split applies our §6 predicate to each part separately. Two claims travel under the word *enumeration*, and our scores separate them: *historical auditability*—what was ever endorsed under the published keys (the census above; bounded by payload retention)—and *current eligibility*—which endorsed workloads could still run, additionally bounded by the 45-day validity and 90-day freshness windows whose enforcement is trust-required (row C4b). Row C3 scores the first; the vendor's "complete set of potentially active workloads" phrasing asserts the second, which inherits the first's limits plus C4b's. What does *not* bound enumeration is raw log scale. The slow paths are the alternatives: `rekor-monitor`'s full scan (the guide warns it "can be quite slow") and naive per-index retrieval. At ≈10.7M entries/day the 90-day freshness window holds on the order of a billion entries; on 2026-11-04 we measured batched retrieval through `log/entries/retrieve` (10 indexes per request; 10 batches, 100 entries) at ≈9.7 entries/sec from an unprivileged vantage point (10.3 s per 100), and at ≈5.0–5.7 entries/sec in a three-run replication from a second vantage point (17.7–20.0 s per 100): the rate is vantage-dependent, and either rate puts the naive scan of that window at an order-of-magnitude ≈3–6 years (the batched-sequential implementation's cost on the naive path, not the enumeration route's). Our original estimate used stipulated, unmeasured rates (500–2000 entries/sec); it is superseded by this measurement and should not be cited. The distinction that survives is narrower than our first pass claimed: the log lets anyone enumerate what the *published* keys have endorsed—while no log can show that nothing was ever endorsed under an *unpublished* key; that is the custody question of §5. The route the census rode on is perishable: it is Rekor's v1 index endpoint, which the v2 design removes (Appendix C)—no v2 equivalent exists, so were the v1 index retired, the census's index-based collection would be live-reproducible no longer; it would be checkable from the archived raw outputs (rechecking archived outputs is not independently repeating the collection), while the routes above remain, unvalidated by us; its live reproducibility lasts only while v1 is served. Were v1 retired, two of the three discovery routes survive, neither riding the index endpoint: the BigQuery dataset, queried by key digest as the guide does, and full-log scans in the `rekor-monitor` manner—at the scan cost measured above, neither count-checked by us. Neither route answers row C3c's question for the scored persona—the scan route is open to it but measured in years, and the BigQuery route is account-gated, hence closed to it, and unmeasured for the cloud-account class—so C3c stays T under every route open to that persona that we evaluated; and if both lapse with v1, the archived raw outputs in the artifact package are the record audits start from.

## 4.5 Retention and freshness

Three time limits govern checkability: the endorsement's validity (45 days for the worked exemplar), the 90-day freshness window in the reference values, and the store's retention (an endeavour of at least one year). Table 5 (Appendix B) collects them.

The guide's February-2025 Gboard policy worked example (endorsement `d72534e0. . .`, policy `a324fe96. . .`) no longer resolves from the transparency store (checked 2026-10-05)—consistent with the documented "at least 1 year" retention. The lapse is not confined to that example (both payloads still returned HTTP 400 on 2026-11-04): across the KMS-layer keys of the §4.1 census a substantial share of older payloads is unserved (24 of 51 stage0, 15 of 43 kernel, 29 of 83 system-image payloads; and one app key, Gboard policy key 2, has all eight of its payloads unserved, none retrievable, its entries and key still public), while recent payloads serve normally—the unserved pattern, consistent with retention expiry, is advanced but uneven across keys/layers (Table 3), a cross-sectional pattern rather than a cliff (we did not stratify availability by entry age across keys; for the exemplar key the five unserved payloads are its five earliest entries, so the timescale rests on the documented

floor, not a measured curve), and is consuming the historical record. One precision on row C5's score, in three parts. The claim scored at row C5 is extracted from the same passage's first sentence—the transparency endpoint "plays an important role in facilitating the discovery [of] the provenance associated with each endorsement". Read as a whole, the passage commits to an endeavour and a floor, not open-ended retention: "we endeavour to keep this endpoint highly available, and to keep all endorsement resources available for at least 1 year from the time of endorsement creation" (inspection guide, availability section [3]). (i) Historical unavailability is a fact, and (ii) it makes the guide's worked example unusable as a route to historical inspection: older payloads, that example among them, are unserved. (iii) No assurance claim is contradicted: the example lapsed at roughly twenty months, past the one-year floor, so not contradicting that text. What the executed check establishes is an observation, not a contradiction: historical payloads lapse past the documented floor, so historical auditability is bounded by the retention the vendor documented. An earlier P* reading, on the ground that the extracted wording states no horizon, is withdrawn at this revision: a horizon's absence in the first sentence does not make an unqualified promise of a passage that states the endeavour and the floor. C5 therefore scores P as an executed observation of bounded historical retrievability. The P grades the facilitation itself, executed over the retained window (430 of 543 attempts hash-matched); the unserved remainder bounds the window, not the facilitation.

## 5 Findings

Across the condensed matrix (Table 1, Appendix B), the 23 vendor-statement rows score 2 V, 10 P and 11 T. The 10 P divide by execution class—five executed-qualified (B2, B3, C3, C5, D1—C5 an executed observation: historical payloads lapse past the documented retention floor, §4.5), four by inspection (A2, B4, C1, C2—the published library read; the retrieved artifacts wire-decoded; the published artifacts read), one partially executed (E1). The matrix's four remaining rows are not vendor statements: both disclosure rows score V, and the audit-derived dependency (C3c, its T conditional on the evaluated routes, §4.4) and the scope record (G4) score T—over all 27 rows, 4 V, 10 P and 13 T, all of them a single auditor's scores. The distribution is an index to the section's result, not the result itself: that result is the residual-trust enumeration below—what each T rests on, and what would decide it. So for the documented exemplar, the central *chain* checks out, at two strengths, stated separately below. The rubric's three components are kept separate at this headline: the letters grade checkability, the division just given is by execution class, and outcomes are the decisions counted next—four of the 27 procedures decided their claims, all with a yes (the V rows; row C5's pre-revision no was withdrawn, §4.5). No V attests runtime behaviour: two are executed artifact verifications (the transparency-log chain; source and build recipe); two are verified statements whose claims are the statements themselves. The full matrix (Appendix B) has 29 rows scoring 4 V / 10 P / 15 T; the two tables reconcile exactly: two merges (A4 into the A3 row; F2 into the B5 row) remove two T rows, arriving at 4 V / 10 P / 13 T over 27 rows; every other row carries the same score in both tables. One extracted conjunct is carried as named residual rather than a row—the production correspondence inside row B1's wording (§3)—counted as a row at its residual grade, the totals would read 4 V / 10 P / 14 T over 28 condensed rows, and the full matrix 4/10/16 over 30 rows. The distribution is sensitive to row construction; we report the variants rather than a fragile count (every cell is tabulated in Table 11, Appendix B). One merge rule applies throughout: a merged row scores P when its parts disagree and at least one part is checkable (V or P), else the common score—so the C3 family (P+T+T) merges to P and the C4 pair (V+T) to P; a merged P so derived asserts a checkable constituent, not a uniform evidentiary status for the composite. Variants are computed on the atomic rows in a fixed order—re-scorings first (A4's flip acts through its merge into A3's row), then merges, then removals—and a re-scoring and a removal of the same row are alternatives, never stacked. *Construction:* C3 merged, 4/10/11 over 25 rows; C4 merged, 3/11/12 over 26; the strict C1/C2 line (both inspection-only rows at T), 4/8/15; weakest-link merges (the weakest part's score) down to 3/9/12 over 24 with both applied; G4 removed, 4/10/12 over 26 (G4 scores no vendor claim; a scope limit on B2); G4 and F2 removed (full matrix), 4/10/13 over 27. *Scorer variance:* the blind automated re-scoring (§3) agreed with all 27 as-executed scores but recorded four rows a competent scorer could land elsewhere on, each an up-flip the adjudication rejected: A3 and A4 at P (row A2's inspection precedent); B2 at V (its executed instance was bit-identical); G4 at P (no vendor claim exists for a procedure to decide). The published scores are the adjudicated ones. *Claim interpretation:* two variants re-read what a claim asserts rather than re-applying the rubric: G2 scored as the disclosure it rests on, 5/10/12 over 27 (not adopted—G2's claim as worded asserts the visibility fact, not its disclosure, so §3's disclosure rule does not reach it, unlike F1 and C4, whose claims are disclosures); and the B2 down-flip (P→T—an author-defined stricter reading, not a scorer output), which reads B2's claim with the production-correspondence conjunct the B1/B2 split carries as residual (Table 9)—not an application of §3's third rule, which governs a score under the claim as worded, and under that claim no procedure was left unexecuted: the rebuild was run. Crossed with the strict C1/C2 line the flip reads 4/7/16. The axes act on disjoint rows, so they stack. The domain's inclusion rule: construction variants (merges, removals); scoring variants crossed in their strict direction only—a stricter reading the row's basis cannot exclude, admitted in the direction the evidence rules force; and the two claim-interpretation variants, as alternative readings of what the claims assert, not score changes. The ‡ cells lie outside the domain (they cross the scorer axis in its lenient direction, crediting evidence no basis contains); the † cells because they record superseded scoring states. The artifact's verifier enumerates all 432 stacks of the tabulated transforms (144 on the condensed base, 288 on the full; the transform options as tabulated above), recomputing each cell from the atomic scores: a computed range, not a selection. Stacking the three strict readings—the C1/C2 line, the B2 flip, the weakest-link C3 merge—reads 4/6/15 over 25 rows; the weakest-link C4 merge on top reads 3/6/15 over 24. Merging the B1/B2 split reads 3/10/13 over 26 rows—outside that declared domain (the tabulated merge axes are the C3 and C4 families), though its uncrossed cell sits within the coordinatewise ranges; crossed with the C4 merges, 2/11/12 (default) and 2/10/13 (weakest-link) over 25 rows; as a domain axis it would extend the verified range to 2–5. Across that joint envelope,

the verified count is 3–5 and the partially checkable count 6–11 (coordinatewise, not jointly attained; within this declared domain—the tabulated axes of Table 11, enumerated by the artifact's verifier—not general robustness bounds): the fragility sits in the P/T split. The amendment-state cells are not envelope members; every sub-3 cell pairs a presentational variant with a reverted score—amendment history, not readings of the present matrix; no cell inside the domain goes below 3. Two scores are amendment-raised (A1, C3): direction-aware follow-ups, frozen before their runs but written after the October results were known (§4); they are confirmatory for their frozen procedures, not blinded confirmations—replications awaiting blinded repetition. One transform is printed—the snapshot (executed against condensed publication matrix): reverting the two amendment-raised scores (A1 to P; the enumeration claim to its pre-split row at T; 25 rows) reads 3 V / 10 P / 12 T; B2's letter is unchanged. The rows are not homogeneous in type (the decomposition at this section's opening): G4, the one row scoring no vendor claim, records the vendor's own reproducibility exclusion, retained as a scope limit on B2 (T).

**What checks out (V).** The log→endorsement→binary→provenance→source chain is real and public for the documented exemplar (retrieved 2026-10-04/08; inclusion-proof verification 2026-10-15 and remaining authentication 2026-11-04, §4.1), at two strengths, stated separately: the log→endorsement→binary edges are hash- and signature-verified end to end, and the rebuild edge was executed on 2026-11-04 (§4.3); the provenance edge is digest equality over provenance statements whose signatures we did not independently verify, which is why row B3 scores P, not V. The chain is exemplar-scoped; for sampled entries beyond it, the verified link is entry→payload (§4.1) only; log membership folds to the signed checkpoint (§4.1; second implementation, 2026-11-04). The build recipe is public and fetched (V). Endorsement keys are public and their fingerprints recompute, but the key row scores P: the Gboard inventory parses to eight keys against the one the 2025 guide's example shows—rotation inferred, not established: the difference does not establish growth, the example not being shown to be the exhaustive 2025 set (its key class is unverified, §4.2). The 90-day freshness window splits: as published, verified (row C4, V); its enforcement inside TEEs unobservable (row C4b, T). The 20 provenance statements are consistent with toolchain determinism, but equally with build caching; input-sensitivity is not established. Side-channel limitations are disclosed in the paper itself—that disclosure is the verified claim (row F1). The "best-effort" TTL (time-to-live; row D3) and the GCP Confidential Space offload path's carve-out from the reproducibility claim ("not guaranteed to be reproducible", row G4) are likewise disclosed in the paper itself; they differ in kind—the TTL states a limit on the vendor's own behaviour, taken on trust (T), while the G4 carve-out is a scope exclusion, not a behaviour claim—retained as a scope record (T†), not an unresolved factual claim.

**What is degraded (P)—and two rescorings.** (1) Enumeration splits into three claims. What the *published* keys have endorsed is enumerable, and we executed it (509,569 entries across 19 reconstructable keys, §4.4): it scores **P**, not V, because two guide fingerprints are unqueryable for lack of key material, so discovery does not cover the guide's full printed key set. "Enumerable" names that discovery result alone: the endpoint returns the entry set for each reconstructable published key; the census's payload hash-matching binds the returns to their entries' digests, and completeness is judged on that returned set, not on whether every payload is still served. Four propositions travel under this row; the P grades the first alone: entry discovery (executed); payload retrieval (retention-limited, §4.5) and workload identification (exhaustive for ≤120-entry keys, sampled for the largest, §4) are observations reported with the census; current eligibility is not scored here (row C4b). Against the vendor's parent claim—that auditors "can know the full set of potentially active binaries and workloads" [1]—the position is this ledger: the census enumerates entry identifiers returned under the reconstructable published keys, not the potentially active set; its remaining constituents stand as just stated—payload readability retention-bounded (§4.5), workload identification exhaustive for ≤120-entry keys and sampled for the five largest (§4), inventory completeness at T (row C3b), index-return completeness at T (row C3c), current eligibility at T (row C4b). Taken as worded, then, the vendor's claim is not decided: discovery scores P, and the complements it needs—inventory completeness, index-return completeness, and current eligibility—score T. The endpoint was cross-checked against the log's signed tree on a seeded sample (24 entries, §4): the returns are authenticated log entries, though a sample drawn from them cannot show the endpoint omits nothing; completeness against the tree is a third dependency (row C3c), scoring **T** on the evaluated credential-free routes (§4.4). Whether the published inventory is *complete*—that nothing devices would accept was endorsed under an unpublished key—is a separate claim (C3b) and scores **T**: no outsider procedure can reach it. The discovery result is time-limited: its index route is removed in the v2 design (§4.4). (2) Live access-policy *content*: policy endorsements are retrievable through the policy keys, and we retrieved and hash-verified them (§4.1); the embedded-program mechanism the system paper describes for Python workloads is present in production policy artifacts—recursive wire-level decoding of the 57 artifacts a fresh retrieval references found complete Python workload programs in five, the Gboard ACT and HMM SentencePiece training programs among them, each with its `_MAX_ZCDP` guard in code (§4.1). Row B4 stays P on the inspection route, not on the sampling bound: the retrieved set was decoded in full (57 artifacts)—the mechanism is present, and the programs are inspectable by recursive wire-format decoding, as here: neither the policy bytes at surface level nor the vendor's string-level inspection path reveals them, and whether the deployed serving path decodes them is not something we test. The decode is inspection-class evidence (§3): it reads artifact content, and inspection caps at P—the line that holds rows A2, C1 and C2—deciding the claim as a statement about the artifacts read, not by an executed check in §6's sense. Sampling bounds population-wide inspectability—a different proposition from the scored one (Table 9)—reported with the census, not charged against this letter. Nor can the 5-of-57 share carry the score: that denominator mixes workload types; the remaining 52 are binary-protobuf configurations. The embedded programs carry their own `_MAX_ZCDP` guard in code; a guard's presence shows the program text self-limits a budget parameter, and we compared each text's constant against the budgets stated in the program texts themselves. The two Gboard texts set the constant at 2.63, and neither states any budget in the text—the quantity the cap guards is computed at runtime from

parameters the texts nowhere assign values—so no comparison is possible for them. The third text (the AGSA/OmniEnt synthesis program) sets the cap at 0.416 and does state its budget inputs—per-query $\varepsilon = 1.0$ and $\delta = 10^{-3}$ over its seven marginal queries—whose composed total ($\rho = 0.236509$) the program itself checks against the cap; it lies under it—an in-text comparison on the inspection route, not a finding about enforcement at runtime. Our October string scan reported non-detection; the decode supersedes it. (3) The reproducible build's place in this list changed kind, not letter: its executed core is verified (the project's CI invocation reproduced the endorsed digest, §4.3)—via an exploratory rescue, not a frozen-protocol confirmation—and row B2 remains P only because the claim as worded covers all endorsed binaries and one was executed; the cross-commit input-sensitivity question rides inside the row's caveats (§4.1).

**The inspection boundary, row by row.** §3's acceptance line decides the inspection-only rows; its application to each is stated here, in one place. A row can score V only when a procedure the scored persona can execute on public artifacts decides the claim as worded against live artifacts; inspection—reading artifact content—decides a claim only as a statement about the artifacts read, and carries P, however complete the reading. Row A2 (the published client library validates the KMS key and the access policy): the evidence is the static reading of the public client code path (2026-11-04)—the code was read, not run against a live endorsement—so the claim is decided as a statement about the published implementation and scores P; that production devices run that library is a separate proposition, scored at row A2b (T). Row C1 (policy pins program and binary digests at four stack layers): the evidence is the published proto and the guide's Listing-2 worked policy, verified as printed; the worked example has lapsed (§4.5) and no served policy's bytes were executed, so the pinning is decided as a statement about the published artifacts—the weakest P class. Row C2 (per-transform access budgets): the budgets are shown in the guides' printed worked example, not read in a served policy's bytes—the same inspection-only ground as C1. Row B4 is held by the same line from the other side of completeness: the retrieved set was wire-decoded in full, but decoding reads artifact content, so the evidence is inspection-class and the row stays P (item (2) above). The line also cuts the other way: execution alone does not reach V when the claim as worded is wider than the executed instance—row B2's executed rebuild is executed-qualified P, because the claim covers all endorsed binaries and one was executed—and no inspection row in this matrix rises to V on the strength of its reading.

**What cannot be verified from outside (T)—the residual.** This enumeration is the audit's core deliverable:

- *Endorsement-key custody.* The log shows key *usage*, never custody. The residual trust in this audit is not a single anchor but a typed set of dependencies—endorsement-key custody (Google), hardware attestation roots (AMD/Intel, row E2), log operation (Sigstore/OpenSSF, under the collusion assumption of §2), and authenticated source distribution (GitHub)—and they fail differently. For the log specifically: the log-integrity scores rest on inclusion against a single signed checkpoint, and cross-checkpoint consistency proofs (append-only consistency between signed checkpoints over time) were not verified (§4.1), so append-only consistency over time is itself part of the residual. For the endorsement key specifically: a compromised Gboard policy key can authorize arbitrary *future* policies, defeating the verifiability chain for data bound under them (and, with operator collusion, the remaining runtime checks as well); it cannot rewrite retained log history, forge the KMS binary's provenance retroactively, or defeat devices' reference-value checks on the KMS itself. Decomposed: the key alone gives signing capability whose policies conforming clients accept as endorsed, binding data and authorizing decryption as designed; KMS reference-value checks, retained history and provenance, and the runtime checks stand under the key alone—the last fall only with the collusion just named. Detection depends on monitors watching the log by endorsement key—which works (`rekor-monitor` or the index endpoint, §4.4)—but only for published keys, and only while payloads are retained. Detection is by discrepancy, not by the log alone: a use counts as compromise evidence when the holder cannot account for it in its own release records; rechecking (below) bounds the uncertain window—it cannot by itself establish that a compromise occurred, and pre-window entries keep their own proofs.
- *Runtime behavior.* That DP noise is actually added as programmed, that the KMS releases keys only to matching TEEs, that results release atomically with pipeline-state updates, that the Noise root↔worker channel holds—all code-inspectable, none externally observable by the no-device persona of §3. Parts are *conditionally* verifiable: a participating device can validate the KMS's attestation against the reference values under row E2's hardware assumptions—outside our persona. Our T therefore reads "no procedure for this persona," not "unverifiable in principle."
- *Sideloaded proprietary logic.* Model architectures and preprocessing protos loaded at runtime are deliberately not in the log. The paper's assurance is that auditors can inspect *how these are used*, rather than the proprietary values themselves. That inspection is not possible from public artifacts: the interface contracts that would show which sideloaded fields can influence data flow are not published, and the artifacts are not in the log, so for our persona the distinction between privacy-relevant and proprietary sideloading is not decidable. Published interface contracts (recommendation (3) in §6) would decide that distinction at the level of the declared interface—which sideloaded fields can influence data flow; whether deployed transforms then comply with the hardcoded characterization would still take a confinement argument that arbitrary sideloaded transforms remain inside the policy's clipping, accounting, and output bounds. Neither is available, so the characterization that all privacy-relevant logic is hardcoded stands as trusted. Two propositions must not be conflated: the proprietary *content*'s privacy—row B5's scored claim, resting on the vendor's characterization—and the confinement of its privacy-relevant *effects* to the hardcoded logic, the claim no

outsider can decide; the absence of an enumerating interface definition is an absence in the §3 claim corpus (system paper, blog post, repository documentation as fetched).

- *Upload-key binding and TTL (row D3).* Two propositions share this row. That uploads are bound to their policy's key is a cryptographic binding—code-inspectable, with its enforcement inside the KMS TEEs and so unobservable. That keys expire after their TTL rests, by the KMS design's own account, on replicated observations of an untrusted time source plus key erasure, and the design states the limitation: elapsed-time assurance, not the binding, is the trust-required part. The two parts share one row under §3's retained-bundle criterion: neither is decidable by an outsider procedure, so no divergence is masked.
- *Hardware roots and side channels.* AMD/Intel attestation roots are trusted, not verified; side-channel observability is acknowledged in the paper itself [1] (see also SNPeek [12]). Bell-Clark et al. [13] identify insider-exploitable side channels in this platform that can circumvent DP guarantees, which sharpens the Adv. 2 residual above: the surface is not hypothetical. (The analysis is Google-first-party; it reports DP as mitigating two of the channels—one in OSS—so the risk applies to the remainder.)
- *Operator views.* "Only metrics and DP weights are visible to the operator" is not checkable without device instrumentation and operator-side audit—though the paper discloses that the operator *does* see which uploads are used per round (§6.3 of Daly et al.), forgoing sampling amplification.

**From checks to the privacy claim.** The matrix scores checkability claim by claim; the advertised guarantee is a composition. Table 8 (Appendix B) maps the obligations the DP guarantee is made of onto what this audit establishes about each. The first two columns are phrased generically, but their transfer to other pipelines is untested.

Against our threat rubric: Adv. 1 (honest-but-curious operator) is substantially constrained on the audited chain by the crypto binding as designed and code-inspected (its in-TEE enforcement is scored at row A3, not here); Adv. 2 (malicious operator) is constrained by the binding as designed and code-inspected, but not eliminated (runtime unobservable, side channels open); Adv. 3 (compromised endorsement key) defeats the verifiability chain for data whose release the key authorizes—it cannot rewrite retained log history or forge the KMS binary's provenance retroactively—and, with operator collusion, the remaining runtime checks as well; its use is detectable via log monitoring by published endorsement key—as a discrepancy against the holder's release records (§5). The consequence rule we apply throughout: endorsements whose log inclusion precedes a compromise stand on their own proofs—inclusion, checkpoint, and endorsement signatures verify without trusting the key's later state—while endorsements first appearing after it must be treated as not trusted solely on that signature, pending authorization reconciliation against the holder's release records; on a suspected compromise, a verifier bounds the window from the log's timestamps and re-checks every chain relying on entries inside it; the window opens where suspicion starts—onset is unobservable, so uncertain-interval entries are re-checked, not presumed clean. Standing on their own proofs is a statement about the proofs, not about authorization: onset may precede suspicion by any interval, and a verifying signature under a later-compromised key does not establish the holder authorized the use—that rests, if at all, on the holder's release records; rechecking inside the window verifies chains and cannot certify the pre-detection interval clean. Three operations must not be conflated: revalidating proofs (chain consistency), assessing authorization against the holder's release records, and determining the potentially affected interval. Log evidence alone cannot bound the third: onset is unobservable, so no defensible lower bound on the affected historical set exists from the log; the suspicion-started window is a recheck window, not a bound. Policy digest-pinning constrains Adv. 5 (malicious workload author) *except* through sideloading, whose contents are not in the log (§5); Adv. 4/6 are carried as trust-required residuals, with no constraining mechanism claimed. Table 6 (Appendix B) collects this mapping.

## 6 Discussion and Recommendations

*For operators claiming "externally verifiable."* (1) Keep the published endorsement-key inventory complete and current, in the channel clients consume, so the audited set cannot drift from the acceptance set unnoticed (two guide fingerprints match no current key). (2) Extend transparency-store retention for *policies* beyond one year, or archive them publicly, content-addressed; otherwise identifiability of the "complete set of workloads" decays on the "at least 1 year" floor. In falsifiable form: an archive is a remedy only if every policy endorsement resolves to an archived payload—the omission check our census runs in sampled form—failing on retention (113 of 543 payloads unserved; the wire-decode retrieval of §4.1 is a separate run, within the enumeration frame of the published keys (rows C3b/C3c)). (3) Publish the sideloading *interface contracts* (which config fields are privacy-relevant) so auditors can check the "hardcoded" claim without seeing proprietary values; the contract supports a check only if it enumerates every field that can influence data flow—compliance still takes the confinement argument of §5—publishing them would make the declared interface decidable from outside, while runtime compliance would remain trust-required. (4) Pin the rebuild's environment in the record: the operator's side is documented—the CI workflow's runner class is 4CPU/16GB/150GB—but our rebuild's host publishes no resource class (§4.3), a gap in this audit's own record; provide a minimal rebuild target, and state the build invocation in the prose documentation as the CI workflow runs it (here it does not resolve at the exemplar-era tree; the working invocation is only in the workflow file). Exercise the target on a schedule, so drift is caught by the operator, not an outside rebuilder; third-party-cloud rebuilders should log the host's environment metadata. (5) Keep an indexed key-lookup route alive for the log's whole life: enumeration is externally verifiable only while entries are findable by endorsement key, and the route our census rode on (Rekor's v1 index) is removed by the v2 design, with no v2 equivalent (§4.4); letting the index lapse converts an executed check into a trust-required one on the operator's deprecation schedule. The index is part of the transparency design: specify and version it as a commitment, not an incidental API surface.

*For the research community.* Treat "externally verifiable" as a testable predicate. *A claim is externally verifiable, relative to the unauthenticated persona of §3, iff that persona can execute a documented procedure on public artifacts and reach a yes/no answer at feasible cost on the routes evaluated.* Decidability in principle—by a procedure no evaluated route makes feasible—is not auditability under this predicate (row C3c, §4.4). A *no* is an answer too: where the executed procedure contradicts the claim as worded, the row records P*—a decisive negative finding, not a partial pass; no row of the matrix records one (row C5's earlier P* was withdrawn at this revision, §4.5). By that test, most extracted claims do *not* fully pass: of the 27 condensed rows, four procedures decided their claims, all with a *yes* (the four V rows: two executed artifact verifications; two statements verified as stated, §5). The 10 P rows divide as in §5; the remaining 13 are trust-required: for the 11 vendor-claim rows and audit-derived row C3c among them, no procedure decides them at feasible scale (row C3c's route is costed in years, §4.4); row G4† scores no vendor claim.

A multi-auditor version would close the reliability gap: extract the claim corpus independently (union-relative recall against the union of the extractions—a bound relative to that union: a claim no extractor captures appears in neither); score from the raw artifacts; freeze the rubric and adjudication rules before scoring; report agreement on the matrix's constitution, disagreements itemized. Ours falls short of it (§3).

*Limitations.* Scope first: a single deployment. The window is 2026-10-04/08, with later re-probes and amendment experiments (Appendix C). Counts differing between October and November (e.g. 106 vs 115 signing-key entries) reflect log growth. We executed no device-side run (row A2's evidence is static inspection of the client code path, §3); the rebuild ran at the exemplar-era commit only, twice by the same exploratory route (§4.3), and statements about the other 19 provenance commits rest on provenance observation, not on rebuilds. The log-integrity scores rest on inclusion against a single signed checkpoint: cross-checkpoint consistency proofs (append-only consistency between signed checkpoints over time) were not verified (§4.1). The claim corpus is a single-auditor extraction from the fixed sources of §3: no second extraction, so its recall is unmeasured—a missed claim would appear neither as a row nor as a T score. Every count here is over the extracted corpus.

## 7 Related Work

The system we audit builds on Eichner et al.'s confidential federated computations design [6]; Cheu et al. [7] extend the same platform toward provably private analytics. Schnabl et al. [8] ("Attestable Audits," TAIG@ICML'25) run safety benchmarks inside TEEs with results published to a transparency log—closest in spirit, but an evaluation prototype, not a training pipeline. Siby et al. [9] (GuaranTEE, EuroMLSys'24) target attestable ML via Arm CCA; Oak's Transparent Release [5] provides the endorsement machinery Google builds on, and Sigstore/Rekor [4] the log. Most on point for our dimension 6, Bell-Clark et al. [13] (Google-first-party) identify side channels in this platform an insider could exploit to circumvent DP—the Adv. 2 residual we score. Industry deployments (Apple PCC [10], Google Private AI Compute [14], Meta Private Processing [15]) publish transparency artifacts; to our knowledge—on a scoping search (web and arXiv sweeps, 2026-10-06/14/17/20), not a systematic review—none has received an outsider audit of its *training*-pipeline verifiability claims; the nearest misses were vendor-engaged assessments of an adjacent inference system and zero-knowledge protocol proposals.

Eskandarian et al. [19] systematized Certificate Transparency's monitor/auditor roles; Meiklejohn et al. [20] showed deployed CT's inclusion promises are rarely verified by relying clients. Where that literature audits certificate logs, we audit an endorsement log binding binaries and policies (our census is a monitor's enumeration). LogPicker [21] strengthens CT logs against log-operator misbehaviour by cross-witnessing—the design answer to the collusion residual we can only enumerate; Rekor has no equivalent, so the residual stands as scored.

## 8 Conclusion

We built a proposed seven-dimension audit checklist for sealed-hardware training pipelines and applied it—as a single auditor, from public artifacts alone—to Google's production TEE-based FL system. For the documented exemplar, the central chain checks out end to end (its V amendment-raised, §4), at two strengths—log and endorsement edges recomputed, provenance at digest equality, the rebuild an exploratory rescue (§4.3); beyond it, every served census payload hash-matched (430 of 543; the 113 unserved are §4.5's retention-consistent pattern). An outsider *can* enumerate what the published keys have endorsed (row C3, amendment-raised; enumerated 2026-11-04 via the v1 index endpoint the v2 design removes, §4.4), but only as completely as the vendor's published key inventory (row C3b) and the index's returns against the log's tree (row C3c)—both trust-required, C3c's on the evaluated credential-free routes and costings of §4.4; and endorsed payloads lapse past the "at least 1 year" floor, so the record decays unless archived. Beyond those limits lies the defined residual—endorsement-key custody, runtime behavior, sideloaded logic, hardware roots. We publish the framework, procedures, and hashes so the next audit is faster—though the census route itself lapses if Rekor's v1 index is retired (§4.4).

## Competing Interests

The author declares no competing interests. No funding was received for this work.

## Acknowledgment

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

## AI Use

The author used the AI tool Muse Spark (version 1.3, Meta) for drafting and editing prose, for writing audit scripts, and for the automated re-scoring exercise reported in §3. The contents are reviewed, edited, and attested for accuracy by the author.

## Open Science

The full audit code, measurement logs, reproduction scripts, and raw outputs—the census returns and the wire-decode records among

them—are archived in the anonymized artifact at https://anonymous.4open.science/r/sealed-hardware-fl-audit-835E/ (available until October 2027), hash-pinned by the artifact's SHA-256 manifest (`SHA256SUMS`, at the artifact's root); the author maintains the anonymized copy in the interim, and a permanent copy will be deposited on acceptance. The wire-decode and client-inspection records are `records/E10-E11-results.md` in the artifact; the index cross-check is `records/E13-results.md`.

## Ethical Considerations

This work audits *public claims using public artifacts*. We accessed only endpoints intended for public use, stayed within rate limits, and did not attempt to access non-public data, probe private infrastructure, contact Google, or interact with any user devices or data. No external ethics review was sought: the work involves no human subjects, no personal data, and no interaction with the operator's systems beyond public endpoints, so we judged that no protocol required external approval. Our findings classify as follows: two auditor errors, corrected in §4 (the index-space reading and the key-index premise); availability observations (payload retention lapses, §4.5); documentation drift (two guide fingerprints match no current key, §4.1); and assurance limitations (the §5 residual). None is a vulnerability in the audited system: no finding shows a mechanism failing against its stated specification, and the two that initially appeared to were our own misreadings. Had any finding met that bar, we would have followed coordinated disclosure before publication. The audit is non-adversarial toward individuals: the entities audited are public organizations, its objects published artifacts. A risk we considered: publishing exact procedures for enumerating endorsements could aid evasion of monitoring—we judge this minimal, since the procedures mirror Google's own published audit guide and disclose nothing an insider could not already do. A second risk is overinterpretation: the census shows what the published keys have endorsed, not that no other endorsements exist (row C3b, trust-required). The document-based juxtaposition of Apple PCC relies on public documentation and a peer-reviewed independent analysis; no Apple systems were probed.

## References

[1] Katharine Daly, Yu Xiao, Zachary Garrett, Brett McLarnon, Jianpeng Hou, Arun Ganesh, Yanxiang Zhang, Noriyuki Takahashi, Haicheng Sun, Yuanbo Zhang, Timon Van Overveldt, and Daniel Ramage. Toward provably private learning from federated data. *arXiv:2609.31494 [cs.CR]*, v4, September 30, 2026. https://arxiv.org/abs/2609.31494.

[2] Katharine Daly and Daniel Ramage. Toward provably private learning from federated data. *Google Research Blog*, October 2, 2026. https://research.google/blog/toward-provably-private-learning-from-federated-data/.

[3] Google Parfait. confidential-federated-compute. *GitHub repository*, 2026. https://github.com/google-parfait/confidential-federated-compute. Docs: `docs/inspecting_endorsements.md`, `docs/inspecting_attestation_records.md`.

[4] Sigstore. Rekor transparency log. https://docs.sigstore.dev/logging/overview/.

[5] Project Oak. Transparent release / endorsement documentation. https://github.com/project-oak/transparent-release.

[6] Hubert Eichner et al. Confidential federated computations. *arXiv:2404.10764*, 2024.

[7] Albert Cheu et al. Toward provably private analytics and insights into GenAI use. *arXiv:2510.21684*, 2025.

[8] Christoph Schnabl, Daniel Hugenroth, Bill Marino, and Alastair R. Beresford. Attestable audits: Verifiable AI safety benchmarks using trusted execution environments. In *Workshop on Technical AI Governance (TAIG) at ICML*, 2025. https://arxiv.org/abs/2506.23706.

[9] Sandra Siby, Sina Abdollahi, Mohammad Maheri, Marios Kogias, and Hamed Haddadi. GuaranTEE: Towards attestable and private ML with CCA. In *Proc. 4th Workshop on Machine Learning and Systems (EuroMLSys)*, 2024. https://arxiv.org/abs/2404.00190.

[10] Apple. Private Cloud Compute security guide, 2024. https://security.apple.com/documentation/private-cloud-compute.

[11] Yannik Dittmar, Marvin Jerome Stephan, Thomas Völkl, Matthias Hollick, and Jiska Classen. Unlocking Apple's Private Cloud Compute: An Analysis of Privacy-Preserving Artificial Intelligence. In *Proc. ACM WiSec*, 2026. https://arxiv.org/abs/2605.24239.

[12] Ruiyi Zhang et al. SNPeek: Side-channel analysis for privacy applications on confidential VMs. In *NDSS*, 2026.

[13] James Bell-Clark, Albert Cheu, Adria Gascon, and Jonathan Katz. Hardening confidential federated compute against side-channel attacks. *arXiv:2603.21469 [cs.CR]*, 2026. https://arxiv.org/abs/2603.21469.

[14] Google. Private AI Compute technical brief, November 2025. https://services.google.com/fh/files/misc/private_ai_compute_technical_brief.pdf.

[15] Meta. Private Processing for WhatsApp overview, June 2025. https://ai.meta.com/static-resource/private-processing-technical-whitepaper.

[16] James Petrie, Onni Aarne, Nora Ammann, and David Dalrymple. flexHEG: Flexible hardware-enabled guarantees for AI compute. *arXiv:2506.15093*, 2025.

[17] Guowen Xu, Hongwei Li, Sen Liu, Kan Yang, and Xiaodong Lin. VerifyNet: Secure and verifiable federated learning. *IEEE Trans. Inf. Forensics Security*, 15:911–926, 2020.

[18] Hengrui Jia, Mohammad Yaghini, Christopher A. Choquette-Choo, Natalie Dullerud, Anvith Thudi, Varun Chandrasekaran, and Nicolas Papernot. Proof-of-learning: Definitions and practice. In *Proc. IEEE Symp. Security and Privacy (S&P)*, 2021. https://arxiv.org/abs/2103.05633.

[19] Saba Eskandarian, Eran Messeri, Joseph Bonneau, and Dan Boneh. Certificate Transparency with Privacy. *Proceedings on Privacy Enhancing Technologies*, 2017(4):232–247. https://doi.org/10.1515/popets-2017-0036.

[20] Sarah Meiklejohn, Joe DeBlasio, Devon O'Brien, Chris Thompson, Kevin Yeo, and Emily Stark. SoK: SCT Auditing in Certificate Transparency. *Proceedings on Privacy Enhancing Technologies*, 2022(3):336–353. https://doi.org/10.56553/popets-2022-0075.

[21] Alexandra Dirksen, David Klein, Robert Michael, Tilman Stehr, Konrad Rieck, and Martin Johns. LogPicker: Strengthening Certificate Transparency Against Covert Adversaries. *Proceedings on Privacy Enhancing Technologies*, 2021(4):184–202. https://doi.org/10.2478/popets-2021-0066.

[22] Open Source Security Foundation. Running Sigstore as a managed service: a tour of Sigstore's public good instance. *OpenSSF Blog*, Oct. 2023. Notes Sigstore began as a collaboration between Red Hat and the Google Open Source Security Team. http://openssf.org/blog/2023/10/03/running-sigstore-as-a-managed-service-a-tour-of-sigstores-public-good-instance/.

## A Artifact Package Contents

All audit procedures ran against public endpoints on 2026-10-04/08, with re-probes on 2026-10-15 and amendment experiments on 2026-11-04/05; no credentials, devices, or private data were used, apart from the §4.3 rebuild's commodity cloud account (verifier class (ii) of §3). Re-execution is time-limited: the census rests on Rekor's v1 index endpoint, which the v2 design removes (Appendix C), and payload availability decays under retention (§4.5). Artifacts: (1) Python scripts for the Rekor/endorsement/provenance chain (`scripts/`); (2) raw fetched artifacts (Rekor entries, endorsement statements, SLSA bundles) with SHA-256 recomputation logs (`data/`); (3) the Bazel build-attempt log (`data/build_kms_attempt.log`); (4) the executed matrix (the audit matrix as executed, 27 rows; `AUDIT_MATRIX.md`)—the full matrix (29 rows) adds two revision-stage split rows (A2b, C3c), recorded in §3 and Appendix B; the condensed publication matrix is a different 27-row set (the full matrix with A4 and F2 merged away; the as-executed set has A4/F2 separate and predates the splits); (5) pilot report (`PILOT_REPORT.md`); (6) the 2026-11-04 census, signature-verification, documented-build, wire-format-decode, client-inspection, and index cross-check records, the frozen amendment, both scoring-reliability records, and the 2026-11-05 cross-route reconciliation record (`records/`, `AMENDMENT-2026-11-04.md`, `AMENDMENT-2026-11-05.md`). The

review artifact's contents are pinned by a SHA-256 manifest (`SHA256SUMS`, covering every file except the bundled manuscript copy, which itself states this hash); the manifest's SHA-256 is `76eb9316752e406cf3c9c79d8c3f4ace8034f5078bc7a27288931c62679202d5`, so its contents are pinnable outside the artifact. For anonymous review these are provided via an anonymized repository link (https://anonymous.4open.science/r/sealed-hardware-fl-audit-835E/); on acceptance they will be published permanently. Endpoints queried: `rekor.sigstore.dev`, `search.sigstore.dev`, `federatedcompute-pa.googleapis.com`, `api.github.com`. Requests were issued sequentially, with no parallelism beyond 1.

*AI-use disclosure.* See the AI Use section (the mandated disclosure under the PoPETs 2027 template); all quantitative claims were recomputed from raw tool outputs by the author, who takes full responsibility for accuracy, originality, and integrity per the PoPETs policy.

## B Full Audit Matrix

The 29-row matrix is printed in Table 2 (claim names abbreviated; per-claim evidence is in `AUDIT_MATRIX.md` in the artifact) and summarized in Table 1; Table 9 prints the claim wordings as scored and Table 10 the second-scorer agreement table. "Listing-2 worked policy" in the row anchors is the worked policy listing of the vendor's inspection guide [3]—a vendor artifact, not a listing of this paper.

## C Reproduction Commands

*Note: endpoints as documented in October 2026; re-probed 2026-11-05. Rekor's v1* `/log/info` *still returns 404 (as of 2026-10-15), but the v1 endpoints used below remain live:* `POST /api/v1/index/retrieve` *with the exemplar key returned its 115 entries, and entry retrieval by UUID returned the entry—so the census of §4.4 is still reproducible live as of the re-probe date. Rekor v2 removes the search index by design, so no v2 equivalent exists; were v1 retired, the census's index-based collection would be checkable only from the archived raw outputs—not independently repeatable—while the BigQuery and scan routes remain in principle (§4.4). Copyable forms are in the artifact's scripts (*`scripts/fetch_live.sh`*, census scripts); the listing wraps long URLs at column width. Row A1's authentication steps are recorded in the artifact's* `records/E8-census-signatures.md` *and* `records/E13-results.md`.

```
# 1. Rekor entry -> endorsed hash
curl -sL \
"https://rekor.sigstore.dev/api/v1/log/entries/\
108e9186e8c5677a4e8e68884126228c18\
dbe566ab43eb0be79b077401149bd340bc29\
5d78e9cd86" \
 | jq -r '.[] | .body' | base64 -d \
 | jq '.spec.data.hash.value'
# 2. Endorsement payload + hash check
H=62e818ac02a0427b9811d08468e286a\
ca2f5c68ca44191a9ab1f86075afe5a87
curl -sL \
"https://federatedcompute-pa.googleapis.com/\
data/transparency/sha2-256:$H" \
 -o end.json && sha256sum end.json   # must equal $H
# 3. SLSA provenance bundles for the endorsed
#    KMS artifact digest (GitHub Attestation API)
curl -sL \
"https://api.github.com/repos/google-parfait/\
confidential-federated-compute/attestations/\
sha256:d14a1852f08b528f9975245f300d44e2500a\
aff9c8a9c5572e0114b7203e3e47"
# 4. CI build (workflow-documented runner class
#    4CPU/16GB/150GB, unproxied net); executed
#    2026-11-04 on Kaggle (~36 min, 3813 actions,
#    exit 0; the executed host's own resource
#    class is unpublished, S4.3; no container
#    or OS image digest was logged for the run);
#    toolchain: bazelisk resolves the kms
#    module's pinned Bazel 8.2.1
#    (kms/.bazelversion at the checkout),
#    container.tar sha256 == d14a1852... above
git clone https://github.com/google-parfait/\
confidential-federated-compute
cd confidential-federated-compute
git checkout \
 fe439180bb24e65ad46e869eddee88f9c9b34875
cd kms
OUT="$HOME/kms-out"
bazelisk run -c opt \
 //:install_release_binaries \
 -- --destdir="$OUT"
sha256sum "$OUT/kms/container.tar"
# 5. All-keys census (per key; counts in Table 3)
curl -s -X POST \
 https://rekor.sigstore.dev/api/v1/index/retrieve \
 -H 'Content-Type: application/json' \
 -d '{"publicKey": "<PEM of endorsement key>"}'
#    (the endorsement keys are the public keys
#    distributed as the reference-values files of
#    the public cfc repository; their SHA-256
#    fingerprints name the rows of Table 3)
# payload for each returned UUID: GET
# /api/v1/log/entries/<UUID>, then fetch
# spec.data.hash.value from the transparency
# store as in (2) and compare sha256.
# Classification: parse JSON statements field by
# field; string-scan every payload + second-level
# artifact for _MAX_ZCDP, ZCDP, def/import/
# python/program markers; then recursively decode
# every policy artifact's protobuf wire format and
# rescan all fields, incl. base64/hex decodings
# (scripts/e10_wire_decode.py in the artifact).
# 6. Retrieval rate + log scale: time 10 batched
# POST /api/v1/log/entries/retrieve calls (10
# indexes each); log size by binary search over
# checkpoint tree sizes on the same endpoint.
# 7. Fallback enumeration route (S4.4), if the
#    v1 index is retired: the BigQuery public
#    dataset "rekor" (Sigstore's mirror of the
#    public Rekor log) is queried by KeyDigest,
#    as the guide itself does; it requires a
#    GCP account. Not executed or count-checked
#    by us (row C3c stands).
```

## D Comparative Case: Apple Private Cloud Compute

Apple PCC is the closest production confidential-computing deployment known to us; we juxtapose it against Google's confidential federated compute (CFC) on the same dimensions, from Apple's security documentation and an independent WiSec'26 analysis [11]. One confound runs through it: PCC is inference-only, CFC a training system, so several dimensions differ by workload type as much as by design. PCC publishes every build to an Apple-operated transparency log, ships a Virtual Research Environment, and devices refuse unlogged builds; *but Apple does not provide reproducible builds* [10, 11], ships partial source only, and claims no source-to-binary correspondence. Google's deployment is broader in claimed

scope (production *training* with DP, full OSS, reproducibility machinery), but its broadest claim—enumerating the complete workload set—rests on key inventory and payload retention; Apple's is narrower—attestation-to-log binding for an inference-only service.

**Table 1: Condensed audit matrix: 27 rows in all—4 V, 10 P, 13 T—of which 26 are scored rows (4 V, 10 P, 12 T) and one, the G4† scope record, scores no vendor claim. Row IDs are the stable identifiers of the full matrix (Table 2); full claim wordings are in Table 9; scores are identical in both tables. Condensation merges A4 into A3 and F2 into B5—both merged rows and both destinations score T, so no aggregation rule affects the totals. V = verified (by outsider execution, or—for disclosure rows—the statement verified as stated, §3); P = partially checkable (sound in principle; degraded, blocked, unexecuted, or inspection-only in practice—the basis states which); T = trust-required (no outsider-executable procedure at feasible scale, §3). The marker † on G4 denotes a row that scores no vendor claim: it records the vendor's own reproducibility exclusion, retained as a scope limit on B2 (§5), not a vendor shortfall. The full 29-row matrix is Table 2; per-row evidence is in the artifact (`AUDIT_MATRIX.md`); the as-executed and condensed 27-row sets differ in composition (Appendix A). C3c's T is conditional on the evaluated routes and rates of §4.4.**

| Dim. | Claim (short) | Score | Anchor |
|---|---|---|---|
| Attestation | **A1** KMS software in public log (exemplar) | V | Rekor idx 1634041292 (virtual) / leaf idx 1512137030; inclusion proof, checkpoint signature, SET, and endorsement signature verified, 2026-11-04 (amendment experiment) |
| Attestation | **A2** Published client library implements device-side validation | P | Executed by static inspection of the public client code path (2026-11-04): the library's verifier checks KMS-key + policy endorsements incl. Rekor inclusion proofs |
| Attestation | **A2b** Production devices run that library | T | Deployment unobservable from public artifacts (§3, rule four) |
| Attestation | **A3** KMS key release only to matching TEEs (+ A4 Noise channel) | T | Inside TEE, unobservable |
| Code | **B1** OSS source + public build recipe | V | Repo + run 26261649715, as of 2026-10-08; the V covers the publicity conjuncts—the built-from conjunct is scored at B2 |
| Code | **B2** Reproducible builds (bit-identical) | P | Project CI invocation executed, 2026-11-04 (exploratory rescue, §4.3): produced container.tar SHA-256 = endorsed digest d14a1852…(exemplar bundle, single commit); other binaries unexecuted |
| Code | **B3** SLSA provenance on Rekor per binary | P | 20 statements via the GitHub Attestation API (signatures not independently verified), as of 2026-10-08; the exemplar's inclusion proof and signatures verified 2026-11-04 |
| Code | **B4** Policy-embedded programs inspectable | P | Bulk-retrieved via policy keys (§4.1 census); wire-level decode: complete Python programs embedded (base64) in 5 of 57 distinct artifacts, incl. the Gboard training programs with `_MAX_ZCDP` in code; the rest binary protobufs |
| Code | **B5** Sideloaded logic stays private (+ F2 future-hardware protection) | T | By design; characterization trusted |
| Policy | **C1** Policy pins program + 4-layer digests | P | Mechanism verified in the published proto + Listing-2 worked policy; live Python-workload policy bytes not executed (guide example lapsed, §4.5); inspection-only (the pinning itself not executed on a live policy's bytes) |
| Policy | **C2** Per-transform access budgets | P | Shown in the guides' printed worked example; not executed on live bytes; inspection-only (the budgets not read in a served policy's bytes) |
| Policy | **C3** Workloads endorsed under published keys enumerable | P | Enumerated 2026-11-04 (amendment experiment): 509,569 entries over 19 reconstructable keys via the index endpoint; the claim splits into C3/C3b/C3c (§4.4); two guide fingerprints unqueryable (no key material); payload readability retention-limited |
| Policy | **C3b** Published key inventory complete | T | No outsider procedure can show deployed acceptance is confined to the published endorsement-key set; the inventory is vendor-asserted and already drifted |
| Policy | **C3c** Index endpoint's returns complete against the log's tree | T | The cross-check authenticates returned entries only; a bounded window reconciliation (2026-11-05 amendment) found no omission in its window (§4.4) |
| Policy | **C4** 90-day freshness window stated | V | Window stated in ref-values and recomputed, as of 2026-10-08 |
| Policy | **C4b** 90-day freshness window enforced | T | Enforcement inside TEEs, unobservable |
| Policy | **C5** Historical policies retrievable | P | $\geq$1-yr retention; 2025 example unserved past the floor; the P grades retained-window facilitation, not lapsed depth |
| Keys | **D1** Endorsement keys published; rotation inferred, not established | P | 8 endorsement + 1 Rekor key published for Gboard (as of 2026-10-08; the 2026-11-04 census frame is Table 3's 19 keys); fingerprints recomputed; growth/rotation not established |
| Keys | **D2** Key custody/compromise response | T | Log shows use, not custody; the consequence rule applied to a compromise is stated in §5 |
| Keys | **D3** Upload keys bound to policy; TTL | T | Binding code-inspectable, enforcement in-TEE; TTL assurance rests |

**Table 2: Full audit matrix (29 rows). Condensing it (A4 into A3, F2 into B5) gives the 27-row condensed matrix (Table 1), under the merge rule of §5; the modifiers * and † are as defined in Table 1's caption. Scores as scored in this paper; the artifact's `AUDIT_MATRIX.md` records the 27-row matrix as executed, with the evidence; rows A2b and C3c are revision-stage splits under the §3 splitting rule. Rows A1, B2, B3, C3, and G2 carry 2026-11-04 rescorings (§4.3, §4.4, §3). Claim names abbreviated; basis phrases are one-line summaries—full evidence text in the artifact. The marker † on G4 denotes a row scoring no vendor claim—a scope limit on B2 (§5), not a vendor shortfall.**

| Row | Claim (short) | Score | Basis (short) |
|---|---|---|---|
| A1 | KMS TEE software recorded in public transparency log (exemplar) | V | Entry, payload, proof, checkpoint + SET + endorsement signatures all verified (amendment experiment) |
| A2 | Published client library validates KMS key + access policy via endorsements + inclusion proofs | P | Static inspection executed 2026-11-04 on the public client library (federated-compute): the attestation verifier checks the upload-key endorsement (KMS keys) and the pipeline configuration (policy keys) via VerifySignedPayload + VerifyRekorLogEntry (inclusion proof recomputed), cross-checking the policy digest in the key endorsement; production devices running it unobservable. P, not V: static inspection under rule four (§3)—the code was read, not run against a live endorsement |
| A2b | Production devices run the published client library's validation | T | Deployment unobservable from public artifacts; the library claim is row A2 (§3, rule four) |
| A3 | KMS releases decryption keys only to TEEs matching policy reference values | T | Enforcement inside KMS TEEs; code-inspectable, runtime unobservable (§3 rule) |
| A4 | Root→worker Noise channel with attestation check | T | The check runs inside the TEEs; the quote chain roots in vendor hardware attestation, and no attestation evidence for the channel is published—no outsider-executable validation; merged into A3 in Table 1 |
| B1 | KMS + data-processing binaries' source public, with public build recipe | V | Repository and recipe fetched and inspected. The V attaches to the publicity conjuncts (source public; recipe public), under §3's companion precedent; the built-from conjunct is scored at row B2 |
| B2 | Binaries reproducibly buildable (bit-identical) | P | Exemplar bundle rebuilt bit-identically (executed, §4.3; exploratory rescue); other binaries unexecuted |
| B3 | SLSA provenance for every endorsed binary on Rekor | P | Retrieved statements assert the matching subject digest on the exemplar; every-binary scope unexecuted; statements' signatures not independently verified |
| B4 | Workload Python programs inspectable (embedded in policy) | P | Executed inspection (wire-level decode, 2026-11-04): 5 of 57 distinct (by digest) artifacts embed a complete Python workload program (base64-encoded; 3 distinct texts: Gboard ACT + HMM SentencePiece training, AGSA DP synthesis), each with its `_MAX_ZCDP` guard in code; 52 are binary configurations with no program text in any scanned form (per-artifact record: `records/E10-E11-results.md`). P: retrieved set decoded in full (57 of 57); the letter grades the inspection route—recursive wire-format decoding only (not surface level, not the vendor's string-level path; serving-path decoding untested). Sampling bounds population-wide readings, not this retrieved-set proposition |
| B5 | Sideloaded proprietary logic stays private | T | Rests on vendor characterization |
| C1 | Policy pins program + binary digests at 4 stack layers | P | Published proto + Listing-2 worked policy verified as printed; live Python-workload policy bytes not executed (guide example lapsed, §4.5); weakest P class—inspection-only: the pinning was read in the published artifacts, |

**Table 2: Full audit matrix (29 rows), continued: rows D1–G4.**

| Row | Claim (short) | Score | Basis (short) |
|---|---|---|---|
| D1 | Endorsement keys published; rotation inferred | P | Fingerprints recomputed; inventory publicly observable; growth/rotation not established |
| D2 | Key custody (who holds the signing keys; compromise response) | T | Custody and response not observable in the log; the consequence rule applied to a compromise is stated in §5 |
| D3 | Upload encryption keys bound to policy; TTL enforced | T | Binding code-inspectable, enforcement in-TEE; TTL assurance rests on the design's stated untrusted-time limitation |
| E1 | Reference values pin stage0 firmware digests + SEV-SNP min SVN | P | Reference values read at `kms/reference_values` (§4.1); full check needs a current served policy's values matched against them |
| E2 | Hardware root of trust (AMD/Intel) intact | T | Hardware roots are vendor trust |
| F1 | Side-channel observability acknowledged as limitation | V | Limitation disclosed in Daly et al. |
| F2 | Sideloaded/proprietary logic protected against malicious operator via future hardware | T | Protection is a future-hardware claim; not testable today; merged into B5 in Table 1 |
| G1 | "Only metrics and DP model weights visible to workload operators" | T | Runtime visibility inside the pipeline unobservable |
| G2 | Operator sees which uploads are used per round (§6.3 of Daly et al.) | T | Disclosure verified in §6.3 of Daly et al.; the visibility fact itself is unobservable |
| G3 | DP noise actually added as programmed; budget not exceeded at runtime | T | Noise addition is a runtime act, unobservable |
| G4 | GCP Confidential Space offload containers (reproducibility excluded) | T† | Explicitly carved out of the reproducibility claim in the vendor's docs ("not guaranteed to be reproducible"); retained as a scope limit on B2, not as a vendor claim |

**Table 3: All-keys census (2026-11-04): entries per published endorsement key via `POST /api/v1/index/retrieve`, and payload verification. Gboard policy keys 3–8 are published keys whose index queries returned no entries. "Exhaustive" = retrieval attempted for every returned entry; "sampled" = 30 evenly spaced entries (31 for Gboard policy 1, including the guide's example). Two further fingerprints printed in the guide (kernel `6052f352...`, initramfs `5f884b69...`) appear in no current reference-values file and could not be queried (no key material); the guide's own examples for those layers embed the current kernel and initramfs keys (rows `aafd4d5a` and `8546b897`). The entry total counts index returns; payloads were verified per-row as marked in the last column, and across the 430 served payloads there were 0 hash mismatches; the Total row's payload counts sum the attempted retrievals (543 = 430 + 113).**

| Key (SHA-256 prefix) | Role | Entries | Payloads: matched / unserved |
|---|---|---|---|
| `98fc8ad4` | KMS stage0 | 51 | exhaustive: 27 / 24 |
| `aafd4d5a` | KMS kernel | 43 | exhaustive: 28 / 15 |
| `8546b897` | KMS initramfs | 44 | exhaustive: 30 / 14 |
| `0b4fcf07` | KMS kernel cmdline | 45 | exhaustive: 31 / 14 |
| `a19ea774` | KMS system image | 83 | exhaustive: 54 / 29 |
| `021205b2` | KMS container key 1 | 3 | exhaustive: 0 / 3 |
| `ea0d1f8f` | Application (container key 2) | 115 | exhaustive: 110 / 5 |
| `3ebd2cd4` | Gboard policy 1 | 30,752 | sampled: 30 / 1 |
| `49140964` | Gboard policy 2 | 8 | exhaustive: 0 / 8 |
| `e5b8b89b` | Gboard policy 3 | 0 | — |
| `48877c9b` | Gboard policy 4 | 0 | — |
| `2c17376f` | Gboard policy 5 | 0 | — |
| `30cab9ee` | Gboard policy 6 | 0 | — |
| `e40567d2` | Gboard policy 7 | 0 | — |
| `927bca59` | Gboard policy 8 | 0 | — |
| `4d41c79b` | Google App policy | 12,051 | sampled: 30 / 0 |
| `ee9911ad` | Pixel Search policy | 4,607 | sampled: 30 / 0 |
| `2bb537ad` | Private Compute Services policy | 409,718 | sampled: 30 / 0 |
| `5aac231c` | Recorder policy | 52,049 | sampled: 30 / 0 |
| Total | | 509,569 | 430 / 113 |

**Table 4: Per-key derivation of the §4 wire-decode sample (row B4's evidence): the six app policy keys of Table 3. Payloads were sampled exhaustively at ≤120 entries, else at 30 evenly spaced entries (31 for Gboard policy 1, including the guide's named entry), fetched and hash-verified; the distinct policy artifacts they referenced were all fetched, hash-verified, and recursively wire-decoded—57 across the six keys. Program-bearing artifacts embed a complete Python workload program (statement references in parentheses). Entries are the index returns at the decode run (2026-11-04)—a fresh draw, distinct from the census sample (§4)—and served counts are that run's own. The log grows between queries (§4.4): for Gboard policy 1 the decode run's returns had reached 30,878 against the census's 30,752 (Table 3), so this table's entry total (509,311) exceeds the six keys' census total (509,185, §4) by exactly that growth; the other five keys' returns are unchanged from the census, and the two tables' totals are not interchangeable. By the decode's classification the 57 divide as five program-carrying artifacts and 52 reference-values bundle configurations with no program text in any scanned form.**

| Policy key | Entries | Attempted | Served | Program-bearing artifacts (refs) |
|---|---|---|---|---|
| Private Compute Services | 409,718 | 30 | 30 | 0 |
| Gboard policy 1 | 30,878 | 31 | 29 | 4 (12) |
| Gboard policy 2 | 8 | 8 | 0 | 0 |
| Google App | 12,051 | 30 | 30 | 1 (8) |
| Pixel Search | 4,607 | 30 | 30 | 0 |
| Recorder | 52,049 | 30 | 30 | 0 |
| Total | 509,311 | 159 | 149 | 5 (20) |

**Table 5: The three time windows (§4.1, §4.2, §4.5).**

| Window | Length | Source |
|---|---|---|
| Endorsement validity | 45 days (worked exemplar; difference; 2026-05-26 → 2026-07-10) | Endorsement payload |
| Freshness window | 90 days; integrated after 2025-02-15 | Reference values |
| Store retention | endeavour: ≥1 year | System documentation |

**Table 6: Per-adversary mapping (from §5 findings). Constraints are as documented and designed; deployment correspondence and runtime enforcement are not established.**

| Adv. | Constrained by | Residual |
|---|---|---|
| 1 | Crypto binding on the audited chain | Substantially constrained |
| 2 | Binding + reproduced bundle (B2) | Runtime unobservable—including whether the reproduced bundle is the deployed one; side channels open |
| 3 | Log monitoring only (detection by discrepancy against the holder's release records, §5) | Defeats the chain for data bound under it; retained history and provenance not rewritable; with operator collusion, the runtime checks too |
| 4 | — | Carried as a T residual (hardware roots trusted) |
| 5 | Policy digest-pinning | Except through sidestanding; whose contents are not logged |
| 6 | — | Carried as a T residual (acknowledged, unobservable) |

**Table 7: Comparator juxtaposition (abbreviated, selected features). PCC = Apple Private Cloud Compute (inference); CFC = Google confidential federated compute (Gboard training). The PCC column is documentation-only (a juxtaposition, not a score); the CFC column combines executed procedures with inspected documentation (see §3).**

| Dimension | CFC (Google) | PCC (Apple) |
|---|---|---|
| Transparency log | Rekor (Sigstore/OpenSSF-operated; Google co-created Sigstore [22]) | Apple-operated [10] |
| Source availability | Full except sidestanded workload artifacts, Apache-2.0 (the sidestanded artifacts are not published, §5) | Partial [10] |
| Reproducible builds | Claimed; one exemplar binary rebuilt bit-identically by us (§4.3) | Explicitly not provided (Apple's guide [10]; independent analysis [11]) |
| Policy language | Budgets; embedded programs observed live (1 of 57 in the sampled retrieved set; guard values not compared to stated budgets, §4.4) | Attestation-vlog only [10] |
| Key custody | Google-held; keys published, rotation inferred, not established (§4.2) | Apple-held; not published [10] |
| Workload | Production training (DP) | Inference only [10] |
| Researcher tooling | Published audit guide | VRE (Virtual Research Environment) + bug bounty [10]; SOC 3 is a program-level audit report, not researcher tooling |

**Table 8: Composition map: the obligations of the advertised DP guarantee, and this audit's status for each. "Inherited" = taken from cited results, not re-proved here; the DP mathematics itself is out of scope (§1).**

| Obligation | Established by | Status here |
|---|---|---|
| Data bound to the named program and binaries | Policy pinning (C1) + endorsement chain for the exemplar (§4.1) | Pinned in the policy as published (proto + worked example, §4.1); enforcement at upload not observed |
| Code identity of the endorsed binary | Executed rebuild (§4.3) | Verified (one binary: rebuilt bundle's SHA-256 = endorsed digest; not deployed code) |
| Sensitivity and clipping as programmed | Embedded program text in retrieved policies (§4.1) | Program text read, not run (B4): the `_MAX_ZCDP` guard read in the program text of the five program-carrying artifacts in the retrieved sample (three distinct texts); sensitivity/clipping values rest on that text alone—not demonstrated for policies outside the sample; runtime enforcement not examinable |
| Noise generation; budget held at runtime | — | Not examinable (G3, T) |
| Output gating; atomic release with state | — | Not examinable (no separate row; runtime residual, §5) |
| DP mechanism's mathematics | Daly et al. [1] | Inherited, not re-proved |
| Leakage channels bounded | Vendor disclosures; Bell-Clark et al. [13] | Partially disclosed (F1); bound unestablished by this audit |

**Table 9: Full claim wordings as scored, with scores. Rows A2b and C3c are revision-stage splits of A2 and C3 under the splitting rule of §3; the remaining wordings are the extracted claim texts of the artifact's `AUDIT_MATRIX.md`. The marker * would denote a negative finding (the executed check contradicted the claim as worded); no row carries it at this revision—row C5's earlier marker was withdrawn (§4.5). † denotes a row that scores no vendor claim: it records the vendor's own reproducibility exclusion, retained as a scope limit on B2 (§5), not a vendor shortfall. Row B1's V covers the claim's publicity conjuncts only; the built-from conjunct is scored at row B2 (Table 2; see B1's row below). The wordings are the scored propositions; a row that decomposes or reformulates a source statement is flagged in the row, here, or under §3's splitting rule.**

| Row | Claim (full wording) | Score |
|---|---|---|
| A1 | KMS TEE software recorded in public transparency log (exemplar) | V |
| A2 | The published client library validates the KMS key and the access policy via endorsements and inclusion proofs | P |
| A2b | Production devices run the published client library's validation | T |
| A3 | KMS releases decryption keys only to TEEs matching policy reference values | T |
| A4 | Root→worker Noise channel with attestation check (merged into A3 in Table 1) | T |
| B1 | Binaries' source public, with a public build recipe—the proposition the V scores [extracted wording: "KMS and data-processing binaries are built from public OSS source, with a public build recipe"; the V covers its publicity conjuncts only—source public; recipe public. The built-from conjunct is scored at row B2 as buildability, not production history; no row verifies that the vendor's production run used this source and recipe. That correspondence is scored by no row—it is deployment residual (T), not B1/B2.] | V |
| B2 | Binaries reproducibly buildable (bit-identical) | P |
| B3 | SLSA provenance for every endorsed binary on Rekor | P |
| B4 | Workload Python programs inspectable (embedded in policy) [the scored proposition is the embedded-program mechanism on the retrieved set, graded on the inspection route the system presents (§5); not population-wide inspectability] | P |
| B5 | Sideloaded proprietary logic stays private | T |
| C1 | Policy pins exact program and root/worker binary digests at four stack layers (firmware/kernel/system/container) | P |
| C2 | Per-transform access budgets (e.g. "at most 1 access") | P |
| C3 | Workloads endorsed under published keys enumerable from the log | P |
| C3b | Published endorsement-key inventory is complete | T |
| C3c | The index endpoint's returned entry set for a published key is complete against the log's tree [audit-derived: no vendor statement asserts it; the completeness dependency presupposed by row C3's claim, separated under §3's splitting rule] | T |
| C4 | Endorsement freshness window stated (90-day window; not-before 2025-02-15) | V |
| C4b | Endorsement freshness window enforced | T |
| C5 | Historical policies retrievable (transparency over time) [scored against the availability passage as a whole: the facilitation sentence read with the endeavour-and-floor sentence that follows it; the executed observation is that payloads lapse past that floor (§4.5)—a limit on historical auditability, not a contradiction of an unqualified claim; the facilitation sentence read alone scored P* at the earlier revision, a reading withdrawn at §4.5] | P |
| D1 | Endorsement keys published; rotation observable | P |
| D2 | Key custody (who holds the signing keys; compromise response) | T |
| D3 | Upload encryption keys bound to policy; TTL enforced | T |
| E1 | Reference values pin stage0 firmware digests and SEV-SNP minimum SVN | P |
| E2 | Hardware root of trust (AMD/Intel) intact | T |
| F1 | Side-channel observability acknowledged as limitation | V |
| F2 | Sideloaded/proprietary logic protected against a malicious operator via future hardware (merged into B5 in Table 1) | T |
| G1 | "Only metrics and DP model weights visible to workload operators" | T |
| G2 | Operator sees which uploads are used per round | T |
| G3 | DP noise actually added as programmed; budget not exceeded at runtime | T |
| G4 | GCP Confidential Space offload containers (reproducibility excluded) [the vendor's own exclusion ("not guaranteed to be reproducible"); a scope record, not a scored claim] | T† |

**Table 10: Second-scorer exercise (2026-11-04): agreement on the 23 condensed rows of the superseded constitution, as then constituted—21 of 23, Cohen's $\kappa = 0.85$ (§3). Both disagreements scored our score too generous—the enumeration row (P vs. T, on the evidence then presented) and the per-round upload-use row (P vs. T)—and both were adopted at the time. The exercise predates the C3/C4 splits and this revision's A2 and C3c splits; its 23 rows are the published condensed matrix (27 less four rows absorbed by the three merges) with the C3 family, C4 pair, and A2 pair each carried as one merged row.**

| | Second: V | Second: P | Second: T | Then-published total |
|---|---|---|---|---|
| Then-published V | 2 | 0 | 0 | 2 |
| Then-published P | 0 | 11 | 2 | 13 |
| Then-published T | 0 | 0 | 8 | 8 |
| Second total | 2 | 11 | 10 | 23 |

**Table 11: The §5 variant cells, tabulated. Counts are V/P/T over the row total in the last column, computed under the merge rule and fixed computation order stated in §5 (re-scorings first, then merges, then removals; a re-scoring and a removal of the same row are alternatives, never stacked). The joint envelope over the unmarked cells—the scorer axis crossed in its strict direction only—is V 3–5, P 6–11, coordinatewise. Cells marked † are amendment-state cells (a reverted score): they record superseded scoring states, are not envelope members, and are the only cells in which the verified count goes below 3. Cells marked ‡ are the rejected up-flip cells of §5: shown for completeness, they lie outside the envelope's strict-direction scorer axis and are likewise not envelope members. Every cell is recomputed from the atomic Table 2 scores by the artifact's verifier, which also enumerates all 432 stacks of the tabulated transforms; the envelope above is their coordinatewise extrema. Stacked cells are computed on the condensed matrix; cells naming the full matrix use it.**

| Variant | V/P/T | Rows |
|---|---|---|
| *Envelope-domain cells (unmarked)* | | |
| Published condensed matrix | 4/10/13 | 27 |
| Published full matrix | 4/10/15 | 29 |
| C3 family merged back | 4/10/11 | 25 |
| C4 pair merged | 3/11/12 | 26 |
| G2 scored as its disclosure | 5/10/12 | 27 |
| Strict C1/C2 line (C1, C2 at T) | 4/8/15 | 27 |
| Weakest-link C3 merge | 4/9/12 | 25 |
| Weakest-link C4 merge | 3/10/13 | 26 |
| Weakest-link merges, both | 3/9/12 | 24 |
| Remove G4 | 4/10/12 | 26 |
| Remove G4 and F2 (full matrix) | 4/10/13 | 27 |
| B2 flip (P→T) × strict C1/C2 line | 4/7/16 | 27 |
| Stacked strict readings (C1/C2 line, B2 flip, weakest-link C3) | 4/6/15 | 25 |
| Stacked strict readings + weakest-link C4 | 3/6/15 | 24 |
| *Rejected up-flip cells (‡; not envelope members)* | | |
| A3 at P × C4 merge‡ | 3/12/11 | 26 |
| B2 at V‡ | 5/9/13 | 27 |
| G4 at P‡ | 4/11/12 | 27 |
| *Amendment-state cells (†; not envelope members)* | | |
| Snapshot (A1, C3 reverted)† | 3/10/12 | 25 |
| Snapshot × G2-disclosure variant† | 4/10/11 | 25 |
| Snapshot × strict line† | 3/8/14 | 25 |
| Snapshot × C4 merge† | 2/11/11 | 24 |
| Snapshot × weakest-link C4 merge† | 2/10/12 | 24 |
| A1 reverted alone × C4 merge† | 2/12/12 | 26 |